

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250280128

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

FERRARA; Simone

METHODS, BITSTREAMS, APPARATUSES, COMPUTER PROGRAMS AND COMPUTER-READABLE MEDIA

Abstract

A method (**1200**) is provided. A configuration block (**1210**) comprising a configuration value (**1220**) is obtained. The configuration block (**1210**) is useable to reconstruct a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality. The second representation is based on a representation of the image at a second, lower level of quality in the tiered hierarchy. An extension configuration block (**1240**) comprising an extension configuration value (**1250**) is obtained. The configuration value (**1220**) and/or the extension configuration value (**1250**) is useable by a decoder to reconstruct the first representation of the image at the first level of quality.

Inventors: FERRARA; Simone (London, GB)

Applicant: V-NOVA INTERNATIONAL LIMITED (London, GB)

Family ID: 81851946

Appl. No.: 18/858686

Filed (or PCT Filed): April 21, 2023

PCT No.: PCT/GB2023/051064

Foreign Application Priority Data

GB	2205928.1	Apr. 22, 2022
----	-----------	---------------

Publication Classification

Int. Cl.: H04N19/154 (20140101); H04N19/44 (20140101)

U.S. Cl.:

Background/Summary

TECHNICAL FIELD

[0001] This invention relates to methods, bitstreams, apparatuses, computer programs and computer-readable media. In particular, but not exclusively, this invention relates to methods, apparatuses, computer programs and computer-readable media for use in the processing of configuration information that relates to residual data useable to reconstruct a representation of an image at a relatively high level of quality.

BACKGROUND

[0002] Compression and decompression of signals is a consideration in many known systems. Many types of signal, for example video, may be compressed and encoded for transmission, for example over a data communications network. When such a signal is decoded, it may be desired to increase a level of quality of the signal and/or recover as much of the information contained in the original signal as possible.

[0003] Some known systems exploit scalable encoding techniques. Scalable encoding involves encoding a signal along with information to allow the reconstruction of the signal at one or more different levels of quality, for example depending on the capabilities of the decoder and the available bandwidth.

[0004] There are several considerations relating to the reconstruction of signals in a scalable encoding system. One such consideration is the amount of information that is stored, used and/or transmitted. The amount of information may vary, for example depending on the desired level of quality of the reconstructed signal, the nature of the information that is used in the reconstruction, and/or how such information is configured.

[0005] Another consideration is the ability of the decoder to process received information accurately and/or reliably. One factor that may affect the reliability of the decoder is the ability of the decoder to handle errors and/or to handle received information that is unexpected, modified and/or unrecognised by the decoder. Another factor in the accuracy and/or the reliability with which the decoder is able to perform reconstruction relates to providing the decoder with up-to-date information indicating how the signal is to be reconstructed.

[0006] Another consideration is the ability of the encoder and/or the decoder to process information efficiently. The efficiency with which the encoder and/or the decoder processes information may be a factor in the performance level of the encoder and/or the decoder.

SUMMARY

[0007] Various aspects of the present invention are set out in the appended claims.

[0008] Further features and advantages will become apparent from the following description of preferred embodiments, given by way of example only, which is made with reference to the accompanying drawings.

[0009] According to an aspect of the invention there is provided a method comprising: [0010] obtaining a configuration block comprising a configuration value, the configuration block being useable to reconstruct a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy; and [0011] obtaining an extension configuration block comprising an extension configuration value, [0012] wherein the extension configuration value, and optionally the configuration value, is useable by a decoder to reconstruct the first representation of the image at the first level of quality.

[0013] A decoder (which may be referred to as an extended decoder) may be configured to process the extension configuration block. In some cases, by processing the extension configuration block, the decoder updates the configuration (indicated by the configuration value) or performs another predetermined action to determine an “extension” (or “extended”) configuration. The decoder may decode in accordance with the extension configuration. In some examples, the extended decoder processes the extension configuration block to override and/or overrun the configuration of the configuration block. In this way, the extension configuration block may be used to perform one or more predetermined actions in relation to the configuration signalled by the configuration block. Examples of such predetermined actions include, but are not limited to, update, modify, adjust, use, overwrite and ignore the configuration signalled by the configuration block. The term “update” as used herein does not imply that the configuration is modified; updating the configuration may result in the same or a different configuration.

[0014] According to another aspect there is provided a bitstream comprising a configuration block and an extension configuration block, wherein the configuration block and the extension configuration block are associated with a common configuration. The configuration block comprises configuration data and the extension configuration block comprises extension configuration data. The bitstream comprises data, such as video data, which, when decoded, is suitable for displaying, e.g. displaying a video or an image, on a screen.

[0015] Preferably, the extension configuration block may be configured to adjust and/or modify a configuration as signalled by the configuration block.

[0016] Preferably, the extension configuration block is separate from the configuration block in the bitstream. In some examples, the extension configuration block follows at least one configuration block in the bitstream, preferably the extension configuration block is at the end of the bitstream.

[0017] According to another aspect of the invention there is provided a method comprising: [0018] obtaining a configuration block comprising a configuration value, the configuration block being useable to reconstruct a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy; and [0019] obtaining an extension configuration block comprising an extension configuration value, [0020] updating the configuration value based on the extension configuration value [0021] wherein the extension configuration value, and the updated configuration value, are useable by a decoder to reconstruct the first representation of the image at the first level of quality.

[0022] According to another aspect there is provided an encoder configured to: generate a configuration block comprising a configuration value, wherein the configuration value indicates a configuration to be selected by a decoder; and [0023] generate an extension configuration block comprising an extension configuration value, wherein the extension configuration value indicates one or more predetermined actions to be performed by a decoder in relation to the selected configuration.

[0024] According to another aspect there is provided a decoder configured to: [0025] obtain a configuration block comprising a configuration value [0026] select a configuration from a plurality of configurations based on the configuration value; [0027] obtain an extension configuration block comprising an extension configuration value; [0028] perform one or more actions in relation to the selected configuration based on the extension configuration value; [0029] reconstruct, using the configuration block, a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy;

[0030] wherein the decoder uses the extension configuration value to reconstruct the first representation of the image at the first level of quality.

[0031] According to another aspect there is provided a decoder configured to: [0032] obtain a configuration block comprising a configuration value; [0033] obtain an extension configuration block comprising an extension configuration value; and [0034] perform decoding based on the extension configuration block and optionally based on the configuration block.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0035] FIG. 1 shows a schematic block diagram of an example of a signal processing system in accordance with an embodiment of the present invention;

[0036] FIGS. 2A and 2B show a schematic block diagram of another example of a signal processing system in accordance with an embodiment of the present invention;

[0037] FIG. 3 shows a schematic diagram of an example of a data processing technique in accordance with an embodiment of the present invention;

[0038] FIG. 4 shows a schematic diagram of an example of entry point data in accordance with an embodiment of the present invention;

[0039] FIG. 5 shows a schematic diagram of an example of a configuration message in accordance with an embodiment of the present invention;

[0040] FIG. 6 is a flow diagram depicting an example of a method in accordance with an embodiment of the present invention;

[0041] FIG. 7 shows a schematic diagram of another example of a configuration message in accordance with an embodiment of the present invention;

[0042] FIG. 8 shows a schematic diagram of another example of a configuration message in accordance with an embodiment of the present invention;

[0043] FIG. 9 shows a schematic diagram of another example of a configuration message in accordance with an embodiment of the present invention;

[0044] FIG. 10 is a flow diagram depicting an example of a method in accordance with an embodiment of the present invention;

[0045] FIG. 11 is a flow diagram depicting another example of a method in accordance with an embodiment of the present invention;

[0046] FIG. 12 shows a schematic diagram of another example of a data processing technique in accordance with an embodiment of the present invention; and

[0047] FIG. 13 shows a schematic block diagram of an example of an apparatus in accordance with an embodiment of the present invention.

DETAILED DESCRIPTION

[0048] Referring to FIG. 1, there is shown an example of a signal processing system **100**. The signal processing system **100** is used to process signals. Examples of types of signal include, but are not limited to, video signals, image signals, audio signals, volumetric signals such as those used in medical, scientific or holographic imaging, or other multidimensional signals.

[0049] The signal processing system **100** includes a first apparatus **102** and a second apparatus **104**. The first apparatus **102** and second apparatus **104** may have a client-server relationship, with the first apparatus **102** performing the functions of a server device and the second apparatus **104** performing the functions of a client device. The signal processing system **100** may include at least one additional apparatus (not shown). The first apparatus **102** and/or second apparatus **104** may comprise one or more components. The one or more components may be implemented in hardware and/or software. The one or more components may be co-located or may be located remotely from each other in the signal processing system **100**. Examples of types of apparatus include, but are not limited to, computerised devices, handheld or laptop computers, tablets, mobile devices, games consoles, smart televisions, set-top boxes, augmented and/or virtual reality headsets etc.

[0050] The first apparatus **102** is communicatively coupled to the second apparatus **104** via a data communications network **106**. Examples of the data communications network **106** include, but are not limited to, the Internet, a Local Area Network (LAN) and a Wide Area Network (WAN). The first and/or second apparatus **102**, **104** may have a wired and/or wireless connection to the data communications network **106**.

[0051] The first apparatus **102** comprises an encoder **108**. The encoder **108** is configured to encode data comprised in the signal, which is referred to hereinafter as “signal data”. For example, where the signal is a video signal, the encoder **108** is configured to encode video data. Video data comprises a sequence of multiple images or frames. The encoder **108** may perform one or more further functions in addition to encoding signal data. The encoder **108** may be embodied in various different ways. For example, the encoder **108** may be embodied in hardware and/or software.

[0052] Although in this example the first apparatus **102** comprises the encoder **108**, in other examples the first apparatus **102** is separate from the encoder **108**. In such examples, the first apparatus **102** is communicatively coupled to the encoder **108**. The first apparatus **102** may be embodied as one or more software functions and/or hardware modules.

[0053] The second apparatus **104** comprises a decoder **110**. The decoder **110** is configured to decode signal data. The decoder **110** may perform one or more further functions in addition to decoding signal data. The decoder **110** may be embodied in various different ways. For example, the decoder **110** may be embodied in hardware and/or software.

[0054] Although in this example the second apparatus **104** comprises the decoder **110**, in other examples, the second apparatus **104** is separate from the decoder **110**. In such examples, the second apparatus **104** is communicatively coupled to the decoder **110**. The second apparatus **104** may be embodied as one or more software functions and/or hardware modules.

[0055] The encoder **108** encodes signal data and transmits the encoded signal data to the decoder **110** via the data communications network **106**. The decoder **110** decodes the received, encoded signal data and generates decoded signal data. The decoder **110** may output the decoded signal data, or data derived using the decoded signal data. For example, the decoder **110** may output such data for display on one or more display devices associated with the second apparatus **104**.

[0056] In some examples described herein, the encoder **108** transmits to the decoder **110** a representation of a signal at a given level of quality and information the decoder **110** can use to reconstruct a representation of the signal at one or more higher levels of quality. Such information may be referred to as “reconstruction data”. In some examples, “reconstruction” of a representation involves obtaining a representation that is not an exact replica of an original representation. The extent to which the representation is the same as the original representation may depend on various factors including, but not limited to, quantisation levels. A representation of a signal at a given level of quality may be considered to be a rendition, version or depiction of data comprised in the signal at the given level of quality. In some examples, the reconstruction data is included in the signal data that is encoded by the encoder **108** and transmitted to the decoder **110**. For example, the reconstruction data may be in the form of metadata. In some examples, the reconstruction data is encoded and transmitted separately from the signal data.

[0057] The information the decoder **110** uses to reconstruct the representation of the signal at the one or more higher levels of quality may comprise residual data, as described in more detail below. Residual data is an example of reconstruction data. The information the decoder **110** uses to reconstruct the representation of the signal at the one or more higher levels of quality may also comprise configuration data relating to processing of the residual data. The configuration data may indicate how the residual data has been processed by the encoder **108** and/or how the residual data is to be processed by the decoder **110**. The configuration data may be signaled to the decoder **110**, for example in the form of metadata.

[0058] Referring to FIGS. 2A and 2B, there is shown schematically an example of a signal processing system **200**. The signal processing system **200** includes a first apparatus **202** comprising

an encoder and a second apparatus **204** comprising a decoder. In each of the first apparatus **202** and the second apparatus **204**, items are shown on two logical levels. The two levels are separated by a dashed line. Items on the first, highest level relate to data at a relatively high level of quality. Items on the second, lowest level relate to data at a relatively low level of quality. The relatively high and relatively low levels of quality relate to a tiered hierarchy having multiple levels of quality. In some examples, the tiered hierarchy comprises more than two levels of quality. In such examples, the first apparatus **202** and the second apparatus **204** may include more than two different levels. There may be one or more other levels above and/or below those depicted in FIGS. **2A** and **2B**.

[0059] Referring first to FIG. **2A**, the first apparatus **202** obtains a first representation of an image at a relatively high level of quality **206**. A representation of a given image is a representation of data comprised in the image. The image may be a given frame of a video. The first representation of the image at the relatively high level of quality **206** will be referred to as “input data” hereinafter as, in this example, it is data provided as an input to the encoder in the first apparatus **202**. The first apparatus **202** may receive the input data **206**. For example, the first apparatus **202** may receive the input data **206** from at least one other apparatus. The first apparatus **202** may be configured to receive successive portions of input data **206**, e.g. successive frames of a video, and to perform the operations described herein to each successive frame. For example, a video may comprise frames F.sub.1, F.sub.2, . . . F.sub.T and the first apparatus **202** may process each of these in turn.

[0060] The first apparatus **202** derives data **212** based on the input data **206**. In this example, the data **212** based on the input data **206** is a representation **212** of the image at the relatively low level of quality. In this example, the data **212** is derived by performing a downsampling operation on the input data **206** and will therefore be referred to as “downsampled data” hereinafter. In other examples, the data **212** is derived by performing an operation other than a downsampling operation on the input data **206**.

[0061] In this example, the downsampled data **212** is processed to generate processed data **213** at the relatively low level of quality. In other examples, the downsampled data **212** is not processed at the relatively low level of quality. As such, the first apparatus **202** may generate data at the relatively low level of quality, where the data at the relatively low level of quality comprises the downsampled data **212** or the processed data **213**.

[0062] In some examples, generating the processed data **213** involves encoding the downsampled data **212**. Encoding the downsampled data **212** produces an encoded image at the relatively low level of quality. The first apparatus **202** may output the encoded image, for example for transmission to the second apparatus **204**. A series of encoded images, e.g. forming an encoded video, as output for transmission to the second apparatus **204** may be referred to as a “base” stream. Instead of being produced in the first apparatus **202**, the encoded image may be produced by an encoding device that is separate from the first apparatus **202**. The encoded image may be part of an H.264 encoded video. Generating the processed data **213** may, for example, comprise generating successive frames of video as output by a separate encoding device such as an H.264 video encoder. An intermediate set of data for the generation of the processed data **213** may comprise the output of such an encoder, as opposed to any intermediate data generated by the separate encoding device.

[0063] Generating the processed data **213** at the relatively low level of quality may further involve decoding the encoded image at the relatively low level of quality. The decoding operation may be performed to emulate a decoding operation at the second apparatus **204**, as will become apparent below. Decoding the encoded image produces a decoded image at the relatively low level of quality. In some examples, the first apparatus **202** decodes the encoded image at the relatively low level of quality to produce the decoded image at the relatively low level of quality. In other examples, the first apparatus **202** receives the decoded image at the relatively low level of quality, for example from an encoding and/or decoding device that is separate from the first apparatus **202**. The encoded image may be decoded using an H.264 decoder. The decoding by a separate decoding

device may comprise inputting encoded video, such as an encoded data stream configured for transmission to a remote decoding device, into a separate black-box decoder implemented together with the first apparatus **202** to generate successive decoded frames of video. Processed data **213** may thus comprise a frame of video data that is generated via a complex non-linear encoding and decoding process, where the encoding and decoding process may involve modelling spatio-temporal correlations as per a particular encoding standard such as H.264. However, because the output of any encoder is fed into a corresponding decoder, this complexity is effectively hidden from the first apparatus **202**.

[0064] In an example, generating the processed data **213** at the relatively low level of quality further involves obtaining correction data based on a comparison between the downsampled data **212** and the decoded image obtained by the first apparatus **202**, for example based on the difference between the downsampled data **212** and the decoded image. The correction data can be used to correct for errors introduced in encoding and decoding the downsampled data **212**. In some examples, the first apparatus **202** outputs the correction data, for example for transmission to the second apparatus **204**, as well as the encoded signal. This allows the recipient to correct for the errors introduced in encoding and decoding the downsampled data **212**. This correction data may also be referred to as a “first enhancement” stream. As the correction data may be based on the difference between the downsampled data **212** and the decoded image it may be seen as a form of residual data (e.g. that is different from the other set of residual data described later below).

[0065] In some examples, generating the processed data **213** at the relatively low level of quality further involves correcting the decoded image using the correction data. For example, the correction data as output for transmission may be placed into a form suitable for combination with the decoded image, and then added to the decoded image. This may be performed on a frame-by-frame basis. In other examples, rather than correcting the decoded image using the correction data, the first apparatus **202** uses the downsampled data **212**. For example, in certain cases, just the encoded then decoded data may be used and in other cases, encoding and decoding may be replaced by other processing.

[0066] In some examples, generating the processed data **213** involves performing one or more operations other than the encoding, decoding, obtaining and correcting acts described above.

[0067] The first apparatus **202** obtains data **214** based on the data at the relatively low level of quality. As indicated above, the data at the relatively low level of quality may comprise the processed data **213**, or the downsampled data **212** where the downsampled data **212** is not processed at the lower level. As described above, in certain cases, the processed data **213** may comprise a reconstructed video stream (e.g. from an encoding-decoding operation) that is corrected using correction data. In the example of FIGS. 2A and 2B, the data **214** is a second representation of the image at the relatively high level of quality, the first representation of the image at the relatively high level of quality being the input data **206**. The second representation at the relatively high level of quality may be considered to be a preliminary or predicted representation of the image at the relatively high level of quality. In this example, the first apparatus **202** derives the data **214** by performing an upsampling operation on the data at the relatively low level of quality. The data **214** will be referred to hereinafter as “upsampled data”. However, in other examples one or more other operations could be used to derive the data **214**, for example where data **212** is not derived by downsampling the input data **206**. It should be noted that the references to relatively high and relatively low levels of quality may correspond to references to a first and second level of quality, where the second level of quality is higher than the first level of quality. As described herein, in certain cases, the levels of quality may correspond to different spatial resolutions.

[0068] The input data **206** and the upsampled data **214** are used to obtain residual data **216**. The residual data **216** is associated with the image. The residual data **216** may be in the form of a set of residual elements. A residual element in the set of residual elements **216** may be associated with a respective image element in the input data **206**. An example of an image element is a pixel.

[0069] In this example, a given residual element is obtained by subtracting a value of an image element in the upsampled data **214** from a value of a corresponding image element in the input data **206**. As such, the residual data **216** is useable in combination with the upsampled data **214** to reconstruct the input data **206**. The residual data **216** may also be referred to as “reconstruction data” or “enhancement data”. In one case, the residual data **216** may form part of a “second enhancement” stream.

[0070] The first apparatus **202** obtains configuration data relating to processing of the residual data **216**. The configuration data indicates how the residual data **216** has been processed and/or generated by the first apparatus **202** and/or how the residual data **216** is to be processed by the second apparatus **204**. The configuration data may comprise a set of configuration parameters. The configuration data may be useable to control how the second apparatus **204** processes data and/or reconstructs the input data **206** using the residual data **216**. The configuration data may relate to one or more characteristics of the residual data **216**. Different configuration data may result in different processing being performed on and/or using the residual data **216**. The configuration data is therefore useable to reconstruct the input data **206** using the residual data **216**.

[0071] In this example, the first apparatus **202** transmits to the second apparatus **204** data based on the downsampled data **212**, data based on the residual data **216**, and the configuration data, to enable the second apparatus **204** to reconstruct the input data **206**.

[0072] Turning now to FIG. 2B, the second apparatus **204** receives data **220** based on (e.g. derived from) the downsampled data **212**. The second apparatus **204** also receives data based on the residual data **216**. For example, the second apparatus **204** may receive a “base” stream (data **220**), a “first enhancement stream” (any correction data) and a “second enhancement stream” (residual data **216**). The second apparatus **204** also receives the configuration data relating to processing of the residual data **216**. The data **220** based on the downsampled data **212** may be the downsampled data **212** itself, the processed data **213**, or data derived from the downsampled data **212** or the processed data **213**. The data based on the residual data **216** may be the residual data **216** itself, or data derived from the residual data **216**.

[0073] In some examples, the received data **220** comprises the processed data **213**, which may comprise the encoded image at the relatively low level of quality and/or the correction data. In some examples, for example where the first apparatus **202** has processed the downsampled data **212** to generate the processed data **213**, the second apparatus **204** processes the received data **220** to generate processed data **222**. Such processing by the second apparatus **204** may comprise decoding an encoded image (e.g. that forms part of a “base” encoded video stream) to produce a decoded image at the relatively low level of quality. In some examples, the processing by the second apparatus **204** comprises correcting the decoded image using obtained correction data. Hence, the processed data **222** may comprise a frame of corrected data at a first or relatively low level of quality. In some examples, the encoded image at the relatively low level of quality is decoded by a decoding device that is separate from the second apparatus **204**. The encoded image at the relatively low level of quality may be decoded using an H.264 decoder.

[0074] In other examples, the received data **220** comprises the downsampled data **212** and does not comprise the processed data **213**. In some such examples, the second apparatus **204** does not process the received data **220** to generate processed data **222**.

[0075] The second apparatus **204** uses data at the relatively low level of quality to derive the upsampled data **214**. As indicated above, the data at the relatively low level of quality may comprise the processed data **222**, or the received data **220** where the second apparatus **204** does not process the received data **220** at the relatively low level of quality. The upsampled data **214** is a preliminary representation of the image at the relatively high level of quality. The upsampled data **214** may be derived by performing an upsampling operation on the data at the relatively low level of quality.

[0076] The second apparatus **204** obtains the residual data **216**. The residual data **216** is useable

with the upsampled data **214** to reconstruct the input data **206**. The residual data **216** is indicative of a comparison between the input data **206** and the upsampled data **214**.

[0077] The second apparatus **204** also obtains the configuration data related to processing of the residual data **216**. The configuration data is useable by the second apparatus **204** to reconstruct the input data **206**. For example, the configuration data may indicate a characteristic or property relating to the residual data **216** that affects how the residual data **216** is to be used and/or processed, or whether the residual data **216** is to be used at all. In some examples, the configuration data comprises the residual data **216**.

[0078] There are several considerations relating to the use of such configuration data. One such consideration is the amount of information that is generated, stored, transmitted and/or processed. The more information that is used, the greater the amount of resources that may be involved in handling such information. Examples of such resources include transmission resources, storage resources and processing resources. Compared to some known techniques, examples described herein allow a relatively small amount of information to be used. This may reduce the amount of data transmitted via the data communications network **106**. The savings may be particularly relevant where the data relates to high quality video data, where the amount of information transmitted in known systems can be especially high.

[0079] Another consideration relating to the use of such configuration data is the ability to adapt to different types of decoding device. For example, a given encoder may transmit information to a plurality of decoders each having different characteristics such as processing and/or storage capabilities. It is desirable for the use of configuration data in image reconstruction to be agnostic to the type and/or capability of decoder device that is used to perform the reconstruction. Compared to some known techniques, examples described herein enable different types of decoders having different operating characteristics to process configuration data reliably.

[0080] A further consideration relating to the use of such configuration data is the amount and/or the relative complexity of processing involved at the encoder and/or the decoder in order to process the configuration data. Compared to some known techniques, examples described herein reduce an amount and/or a complexity of processing performed by the encoder and/or the decoder in order to obtain and/or process the configuration data. Simplifying the processing performed by the encoder and/or the decoder increases an efficiency of the encoder and/or the decoder.

[0081] Other considerations include the ability of the decoder to perform image reconstruction accurately and reliably. Performing image reconstruction accurately and reliably may affect the ultimate visual quality of the displayed image and consequently may affect a viewer's engagement with the image and/or with a video comprising the image. One factor that may influence the accuracy and/or the reliability with which the decoder can perform reconstruction is in providing the decoder with up-to-date and/or accurate information relating to how reconstruction is to be performed. For example, configuration data may change over time and/or change between different images. If the decoder is not informed of such changes promptly, the resulting visual quality of reconstructed images may be affected. Compared to some known techniques, examples described herein allow configuration information to be signaled relatively frequently to the decoder, thereby enabling the decoder to use relatively up-to-date configuration information in image reconstruction.

[0082] Error handling is also a consideration. For example, the ability of the decoder to handle potential errors in the processing of configuration data may be a factor in the reliability of the decoder. Compared to some known techniques, examples described herein allow a decoder to handle potential errors as well as the receipt of unexpected or unrecognized data. This may effectively increase the reliability of decoder.

[0083] Referring to FIG. 3, there is shown schematically an example of a data processing technique **300**. The data processing technique **300** may be performed by an apparatus comprising an encoder such as the first apparatus **102** described above. The data processing technique **300** involves performing byte-wise processing of configuration data, as described in more detail below.

[0084] Configuration data **310** is obtained. The configuration data **310** is related to processing of residual data, for example residual data **216** as described above. It will be understood that configuration data **310** as depicted in FIG. **3** is an illustrative representation of the content of such configuration data. In practical use, configuration data may be read and/or written as a series of 1s and 0s, for example [0 1 0 0 0 1 0].

[0085] The configuration data **310** comprises a set of values **320**. The set of values **320** corresponds to a set of configuration parameters. The value of a given configuration parameter has a given bit-length. The bit-length is the number of bits that are used to represent the value of the given configuration parameter. The bit-length of a value of a given configuration parameter may be fixed or may be variable. For a variable bit-length, the number of bits that are used to represent the value of the given configuration parameter varies based on the value of the given configuration parameter.

[0086] In some examples, the configuration data **310** comprises a reference to a look-up table that stores possible values of a given configuration parameter. For example, the configuration data **310** may comprise a value that is useable to obtain the value of the configuration parameter from the look-up table.

[0087] The configuration data **310** is arranged into a sequence of bytes **330**. The sequence of bytes **330** comprises an integer number of bytes. In this example, the values of the configuration parameters **320** are arranged into a sequence of bytes comprising 5 bytes, although it will be understood that other numbers of bytes may be used in other examples. In this example, a given byte in the sequence of bytes **330** comprises 8 bits. The sequence of bytes **330** may be part of a larger byte sequence or byte-wise stream. Arranging the configuration data **310** into a sequence of bytes **330** enables byte-wise processing to be performed on the configuration data **310**. In byte-wise processing, data is processed on a byte-by-byte basis.

[0088] Processing the configuration data **310** in a byte-wise manner may be more efficient than processing the configuration data **310** in a bit-wise manner. Bit-wise processing may involve tracking which bit within a given byte is the next bit to be read or written. When a group of bits that represents a given parameter straddles a byte boundary, additional processing may be performed in order to read the group of bits and obtain the given parameter. Such processing may be reduced in a case where byte-wise processing is performed. In some examples, a number of execution cycles may be reduced when processing data in a byte-wise manner compared to a bit-wise manner.

[0089] In some examples, the sequence of bytes **330** is written to memory. Less processing may be involved in storing the configuration data **310** in memory and/or in retrieving the configuration data **310** from memory if the configuration data **310** is handled in a byte-by-byte manner compared to a case in which the configuration data **310** is handled in a bit-by-bit manner. The smallest unit of addressable memory may comprise one byte. As such, information may be stored in memory in a byte-wise manner. Therefore, fewer steps may be involved in storing the configuration data **310** in memory and/or in retrieving the configuration data **310** from memory if the configuration data **310** is packaged into a sequence of bytes. An amount of padding used to package the configuration data **310** into addressable memory units may also be reduced by providing the configuration data **310** to the memory in a byte-wise form.

[0090] Furthermore, transmission of the configuration data **310** via a network such as the Internet may be performed more efficiently if the configuration data **310** is arranged into a sequence of bytes. Data may be transmitted over the Internet via the Transmission Control Protocol (TCP). TCP operates in a transport layer of a protocol stack. TCP obtains data from a stream, packages the data into TCP segments and uses an internet module, for example the Internet Protocol (IP), to transmit each TCP segment to a recipient via the Internet. The data stream may be obtained from memory. A TCP segment may comprise an integer number of bytes. As such, less processing may be involved if the TCP obtains the data that is to be packaged into the TCP segments from a byte-wise stream than from a bit-wise stream. An amount of padding used to package the data into TCP segments

may also be reduced by providing the data to the TCP in a byte-wise form.

[0091] In this example, arranging the configuration data **310** into the sequence of bytes **330** is performed at an application layer of a protocol stack. As such, the application layer may process the configuration data **310** in a byte-by-byte manner instead of a bit-by-bit manner.

[0092] In some examples, the configuration data **310** comprises values of a plurality of configuration parameters that have different bit-lengths. The configuration data **310** may be packed into the sequence of bytes **330** by arranging the values in a predetermined order. The predetermined order may be a different order than an order in which the values are obtained. As such, the values may be re-arranged.

[0093] The values of the configuration parameters **320** may be arranged such that a given byte in the sequence of bytes **330** is filled by the values represented by the different bit-lengths. By filling a given byte in the sequence of bytes **330** with values represented by different bit-lengths arranged in a predetermined order, an amount of padding used in the sequence of bytes **330** may be reduced. As such, data may be packed more efficiently into the sequence of bytes **330** compared to a case in which the values represented by the different bit-lengths are not arranged in the predetermined order. Packing data more efficiently into the sequence of bytes **330** may reduce a number of bytes that are used to store the configuration data **310**. An amount of information that is to be transmitted over a network may therefore be reduced.

[0094] The sequence of bytes **330** is output for processing by a decoder, such as decoder **110**, to enable the decoder to reconstruct the input data **206** using the configuration data **310**.

[0095] Referring to FIG. **4**, there is shown schematically an example of entry point data **400**. Entry point data **400** may be output by encoder **108** and received by decoder **110**. Entry point data **400** is a type of configuration data. In some examples, entry point data **400** forms part of configuration data **310**. Entry point data **400** may be read by the decoder **110** before other configuration data is read. As such, entry point data **400** may be the first configuration data that is processed by the decoder **110** for a given received video stream.

[0096] Entry point data **400** comprises a version parameter **420**. The version parameter **420** may indicate a version number. The version may be a version of a syntax according to which configuration data is arranged. The syntax may be considered a framework that defines how configuration data is to be processed and/or defines the configuration parameters that are to be included in the configuration data. The encoder **108** and/or the decoder **110** may be configured to operate in accordance with the version of the syntax indicated by the version parameter **420**.

[0097] The value of the version parameter **420** has a fixed bit-length. In some examples, the value of the version parameter **420** has a bit-length of 8 bits. As such, the value of the version parameter **420** may be stored in a fixed-length element in the sequence of bytes **330**. A fixed-length element comprises a predetermined integer number of bytes.

[0098] In this example, entry point data **400** comprises an extended version parameter **425**. The value of the extended version parameter **425** has a variable bit-length. The value of the extended version parameter **425** may be stored in a variable-length element in the sequence of bytes **330**.

[0099] A variable-length element comprises an integer number of bytes in the sequence of bytes **330**. The variable-length element may be referred to as a “multi-byte” element, although in some examples the variable-length element comprises a single byte. The one or more bytes of the variable-length element may be contiguous in the sequence of bytes **330**. In some examples, the one or more bytes of the variable-length element are non-contiguous in the sequence of bytes **330**.

[0100] At least one given byte of the variable-length element has one or more predetermined bits arranged to indicate whether or not the variable-length element comprises one or more additional bytes to the at least one given byte. In some examples, the one or more predetermined bits comprises a single predetermined bit. In other examples, the one or more predetermined bits comprises more than one predetermined bit. In some examples, each byte of the variable-length element comprises the one or more predetermined bits. In other examples, at least one byte of the

variable-length element does not comprise the one or more predetermined bits.

[0101] In a case where the one or more predetermined bits comprises a single predetermined bit, the single predetermined bit may be considered a flag that indicates whether or not the one or more additional bytes are present in the variable-length element. For example, if the variable-length element contains an additional byte to the at least one given byte, the value of the predetermined bit may be set to 1. If, on the other hand, the variable-length element does not contain an additional byte to the at least one given byte, the value of the predetermined bit may be set to **0**. In other words, the predetermined bit indicates whether the variable-length element extends beyond the at least one given byte.

[0102] The one or more predetermined bits may be arranged at an end of the at least one given byte. For example, the one or more predetermined bits may comprise one or more last bits of the at least one given byte. In some examples, the one or more predetermined bits comprises one or more first bits of the at least one given byte. In some examples, each of the at least one given byte comprises seven bits arranged to store the value of the extended version parameter **425** and one bit arranged to indicate whether or not the variable-length element comprises one or more additional bytes to the at least one given byte.

[0103] As such, the value of the extended version parameter **425** may be represented using the variable-length element regardless of the bit-length of the value of the extended version parameter **425**. The variable-length element comprises the minimum number of bytes that are useable to store the value of the extended version parameter **425**. The use of the variable-length element may therefore reduce an amount of information that is stored, processed and/or transmitted compared to a case in which a fixed-length element is used that has a size that corresponds to a maximum conceivable bit-length of the value of the extended version parameter **425**. The variable-length element enables the number of possible values that can be used to represent the extended version parameter **425** to be increased as necessary and in a flexible manner. As such, the variable-length element may be used to provide for future developments and/or modifications to the configuration data **310**, for example by allowing for an increase in the bit-length of the value of the extended version parameter **425**, without the need for relatively complex modifications to the structure of the configuration data **310**.

[0104] In this example, the extended version parameter **425** is used selectively. This is depicted using dashed lines in FIG. **4**. The extended version parameter **425** is used if it is determined that the fixed-length version parameter **420** is insufficient to store the value of the version number. In some examples, the value of the version number is stored in the extended version parameter **425** in response to determining that the value of the version number has a predetermined relationship with a predetermined threshold value. In some examples, the predetermined relationship comprises the value of the version number being greater than or equal to the predetermined threshold value. In some examples, if it is determined that the value of the version number does not have the predetermined relationship with the predetermined threshold value, the value of the version number is stored in the version parameter **420** instead of in the variable-length extended version parameter **425**. Using a variable-length element selectively reduces an amount of information that is used, stored and/or transmitted compared to a case in which the variable-length element is used regardless of the value of the version number.

[0105] In some examples, the predetermined threshold value is related to the bit-length of the fixed-length element, namely the bit-length of the version parameter **420**. For example, if the fixed-length element comprises eight bits, values between 0 and 255 may be represented by the fixed-length element. The predetermined threshold value in such a case may be 255, for example. As such, if the value of the version number is greater than 255, the variable-length extended version parameter **425** may be used instead of or in addition to the fixed-length version parameter **420** to store the value of the version number.

[0106] In some examples, data is stored in the version parameter **420** to indicate that the version

number is stored in the extended version parameter **425**. In some examples, the data comprises at least one predetermined bit in the version parameter **420**. The data stored in the version parameter **420** enables the decoder **110** to determine that the value of the version number is stored in the extended version parameter **425**.

[0107] The above-described use of variable-length elements and/or selective inclusion of such variable-length elements may also apply to other configuration parameters of configuration data output by the encoder **108**.

[0108] Additionally, or alternatively to selectively including configuration parameters corresponding to variable-length elements, the encoder **108** may selectively include configuration parameters corresponding to fixed-length elements. In some examples, the configuration data generated by the encoder **108** comprises a parameter presence indicator. The parameter presence indicator specifies whether or not a given set of configuration parameters are comprised in a given part of the configuration data. The given set of configuration parameters has a predetermined format. In the example relating to the version number described above, the parameter presence indicator comprises the predetermined bit in the version parameter **420** that indicates that the extended version parameter **425** is being included. Parameter presence indicators correspond to nested logical statements such as IF statements. That is, if a parameter presence indicator has or is given a particular value, the given set of configuration parameters are included in the configuration data. Including the given set of configuration parameters selectively reduces an amount of information that is used, stored and/or transmitted compared to a case in which the given set of configuration parameters are not selectively included.

[0109] Referring to FIG. 5, there is shown schematically an example of a configuration message **500**. The configuration message **500** may form part of configuration data **310**. The configuration message **500** may be generated by encoder **108**. The configuration message **500** may be received and/or read by decoder **110** after entry point data **400** has been received and/or read.

[0110] The configuration message **500** comprises a header part **510** and a payload part **520**. The payload part **520** comprises a set of configuration parameters **550**. The set of configuration parameters **550** are related to processing of residual data, for example residual data **216** as described above. In some examples, the configuration message **500** is an integer number of bytes long.

[0111] The header part **510** comprises a message type parameter **512**. The message type parameter **512** specifies a message type of the configuration message **500**. The message type specified by the message type parameter **500** may indicate how the configuration message **500** is to be processed. Example types of configuration message are described below.

[0112] The message type parameter **512** is indicative of the payload part **520** having a given payload format. The payload format of the payload part **520** may be determined based on the message type of the configuration message **500**. The payload format may define a set of configuration parameters that are comprised in the payload part **520**. In some examples, the payload format of the payload part **520** defines a particular processing logic that specifies how the content of the payload part **520** is to be processed. The payload part **520** has a format size. The format size of the payload part **520** corresponds to the payload format of the payload part **520**. The format size of a payload part having a given payload format is predetermined or can be determined from the content of the payload part. For example, the format size may be determined based on the combined size of the individual configuration parameters that are defined by the payload format. The format size may be considered to be a possible size or a suggested size of the payload part **520**.

[0113] The header part **510** also comprises a payload size parameter **514**. The payload size parameter **514** specifies the actual size of the payload part **520**. The actual size of the payload part **520** may be an integer number of bytes.

[0114] In some examples, the value of the payload size parameter **514** is stored in a fixed-length element. The fixed-length element storing the value of the payload size parameter **514** may

comprise a predetermined number of bits in a given byte of the header part **510**. For example, the value of the payload size parameter **514** may be stored in a fixed-length element comprising three bits in a one-byte header part.

[0115] In some examples, the payload size parameter **514** comprises a reference to a payload size look-up table that stores possible values of the actual size of the payload part **520**. For example, the payload size parameter **514** may comprise a value that is useable to obtain the actual size of the payload part **520** from the payload size look-up table.

[0116] In some examples, the value of the payload size parameter **514** is stored in a variable-length element. This may be the case, for example, when the actual size of the payload part **520** is not one of the values specified in the payload size look-up table. The actual size of the payload part **520** specified by the payload size parameter **514** may have a variable bit-length. As such, the payload size parameter **514** may be represented using the variable-length element regardless of the bit-length of the payload size parameter **514**. The variable-length element comprises the minimum number of bytes that are useable to store the value of the payload size parameter **514**.

[0117] In some examples, data is stored in a fixed-length element that indicates that the value of the payload size parameter **514** is stored in a variable-length element. For example, if it is determined that the value of the payload size parameter **514** is one of a predetermined set of values, the value may be stored in the fixed-length element and/or the fixed-length element may include a reference to the payload size look-up table. If, on the other hand, it is determined that the value of the payload size parameter **514** is not one of the predetermined set of values, a variable-length element may be used to store the value, and the fixed-length element may be arranged to indicate, for example via a flag comprised in the fixed-length element, that the variable-length element is being used to store the value.

[0118] In this example, the header part **510** contains only the message type parameter **512** and the payload size parameter **514**. In other examples, the header part **510** comprises one or more further parameters. In some examples, the header part **510** is one byte long. For example, the message type parameter **512** may be represented by five bits and the payload size parameter **514** may be represented by the remaining three bits of the one-byte header part **510**. In other examples, for example where a variable-length element is used to represent the actual size of the payload part **520**, the header part **510** comprises more than one byte.

[0119] The actual size of the payload part **520** may be the same as or different from the format size of the payload part **520**. Regardless of whether or not the actual size and the format size are the same, the payload size parameter **514** specifying the actual size is included in the header part **510** of the configuration message **500**.

[0120] Including a payload size parameter **514** that specifies the actual size of the payload part **520** when a possible size of the payload part **520**, namely the format size, can be inferred from the message type parameter **512** may appear to be inefficient and/or a duplication of information, particularly when the format size and the actual size are the same. Including a payload size parameter **514** in the configuration message **500** involves a greater amount of data being used for the configuration message **500** compared to a case in which no such parameter is included.

However, including the payload size parameter **514** in the configuration message **500** facilitates more efficient and reliable processing of the decoder **110** and/or improved error handling compared to a case in which no such parameter is used, as will be described in more detail below.

[0121] Referring to FIG. **6**, there is shown an example of a method **600** of processing configuration data. The method **600** may be performed by an apparatus comprising a decoder such as the second apparatus **104** described above.

[0122] In some examples, residual data is received. In this example, the received residual data is residual data **216** described above. The residual data **216** may be received from an encoder such as encoder **108**.

[0123] At item **610**, a configuration message is received. In this example, the received

configuration message is configuration message **500** described above. The configuration message **500** may be received from encoder **108**.

[0124] At item **620**, the received configuration message **500** is processed using the message type parameter **512** and the payload size parameter **514**. The configuration message **500** is processed to obtain the set of configuration parameters **550** stored in the payload part **520** of the configuration message **500**.

[0125] In some examples, processing the configuration message **500** comprises parsing the header part **510** of the configuration message **500** to obtain the message type parameter **512** and the payload size parameter **514**. Based on the payload size parameter **514**, the actual size of the payload part **520** is determined. Based on the message type parameter **512**, the message type of the configuration message **500** is determined. The configuration message **500** is processed in accordance with the determined message type and the determined actual size of the payload part **520**.

[0126] Processing the configuration message **500** comprises parsing the payload part **520** to obtain the configuration data **550**. In some examples, processing the configuration message **500** comprises tracking the number of bytes of the configuration message **500** that have been read as the payload part **520** is parsed. The decoder **110** may detect the occurrence of an under-read if there is a determined discrepancy between the number of bytes read when parsing the payload part **520** and the actual size of the payload part **520** as specified by the payload size parameter **514**. An under-read may be a potential source of errors when processing configuration messages.

[0127] In some examples, the payload size parameter **514** is used to determine the start of a following configuration message. The following configuration message may be a next successive configuration message relative to configuration message **500** in a sequence of configuration messages received by the decoder **110**. The sequence of configuration messages may form part of configuration data. Configuration message **500** and the following configuration message may correspond to the same image or to different images. For example, configuration message **500** may correspond to a first image in an image sequence and the following configuration message may correspond to a second, different image in the image sequence.

[0128] In some examples, processing the configuration message **500** comprises discarding at least part of the content of the payload part **520**. The at least part of the content of the payload part **520** may be discarded using the payload size parameter **514**. For example, the decoder **110** may skip to the next configuration message, the start of the next configuration message having been determined using the payload size parameter **514**, without processing at least part of the content of the payload part **520**.

[0129] Using the payload size parameter **514** therefore enables the decoder **110** to handle possible under-reads in an efficient manner. Using the payload size parameter **514** to locate the start of the following message also enables the decoder **110** to maintain a correct alignment or synchronisation with a bytestream comprising multiple configuration messages. For example, even if the content and/or format of a given configuration message is unrecognised, unexpected and/or has been updated without the knowledge of the decoder **110**, the decoder **110** is still able to locate the start of the following configuration message and thus remain synchronised with the bytestream.

Furthermore, if a bytestream becomes malformed, the decoder **110** is able to detect when an over-read is about to occur and respond accordingly. For example, the decoder **110** may abort processing of the current configuration message and reset the bytestream to the determined location of the start of the next configuration message. Processing the configuration message **500** using the payload size parameter **514** therefore improves the reliability and/or adaptability of the decoder **110** compared to a case in which such a parameter is not used, in that the decoder **110** is able to handle configuration messages that have been modified and/or that are unrecognised and/or unexpected. The decoder **110** is also able to handle potential under-reads and over-reads without falling out of synch with the bytestream, thereby improving the ability of the decoder **110** to handle potential

errors.

[0130] In some examples, processing the configuration message **500** comprises parsing first content of the payload part **520**. The first content corresponds to the format size of the payload part **520**. The first content may comprise the set of configuration parameters **550**. The first content may comprise first configuration data. Processing the configuration message **500** may further comprise discarding the remaining content of the payload part **520** using the payload size parameter **514**. The remaining content of the payload part **520** comprises content of the payload part **520** that is not comprised in the first content. The remaining content comprises further configuration data. The presence of the remaining content may be determined by detecting a discrepancy between the format size and the actual size of the payload part **520**. In other words, if it is determined that the actual size is greater than the format size, it is determined that additional content, beyond the content that is expected based on the format size, is present in the payload part **520**.

[0131] At item **630**, input data **206** is reconstructed using the obtained set of configuration parameters **550**. The input data **206** may be reconstructed by using the residual data **216** in accordance with the obtained set of configuration parameters **550**.

[0132] In some examples, the payload part **520** of configuration message **500** comprises a first and a second payload portion. The payload size parameter **514** may be indicative of a total size of the first and the second payload portions. The first and the second payload portion may comprise first and second configuration data, respectively. In some examples, the second configuration data comprises supplementary configuration data. The supplementary configuration data is arranged to supplement the first configuration data. The second payload portion may be arranged subsequently with respect to the first payload portion. As such, the first payload portion may be received, processed and/or parsed by the decoder **110** before the second payload portion is received, processed and/or parsed.

[0133] In some examples, the header part **510** does not specify the size of the first payload portion and/or the size of the second payload portion. In some examples, the size of the first payload portion may be determined based on the format size indicated by the message type parameter **512**. As such, the format size indicated by the message type parameter **512** may reflect the content of the first payload portion but not the content of the second payload portion.

[0134] The first configuration data relates to a first set of one or more decoders. The first set of one or more decoders has a first operating characteristic. An example of an operating characteristic is a processing capability. The second configuration data relates to a second set of one or more decoders. The second set of one or more decoders has a second operating characteristic. The decoder **110** to which the configuration message **500** is transmitted may be comprised in the first set or the second set. The encoder **108** that generates the configuration message **500** may be unaware which set of decoders the decoder **110** is comprised in. The encoder **108** may transmit the configuration message **500** to multiple decoders across both sets of decoders.

[0135] In some examples, the first operating characteristic corresponds to a first version according to which the first set of one or more decoders is configured to operate, and the second operating characteristic corresponds to a second, different version according to which the second set of one or more decoders is configured to operate. The second version may be a later version relative to the first version. The first and second versions may be different versions of the syntax framework that defines how configuration messages are to be processed and/or that defines the content of such configuration messages. Therefore, when the syntax is updated from the first version to the second version, the second configuration data is added to a configuration message that contains the first configuration data from the first version. The second configuration data may, for example, be appended to the first configuration data.

[0136] Adding the second configuration data to a configuration message of an existing message type instead of generating a new message type to accommodate the second configuration data reduces an amount of data that is stored, transmitted and/or processed compared to a case in which

new message types are generated. By modifying the configuration of existing message types instead of defining new message types, the total number of configuration messages used to convey all relevant configuration data may be reduced. Each configuration message has a corresponding header which may be at least one byte long. Therefore, by reducing the overall number of configuration messages, the amount of data that is stored, transmitted and/or processed may be reduced.

[0137] In some examples, the decoder **110** is not able to process both the first and the second payload portion. For example, the encoder **108** may have undergone an update according to which the second configuration data is included in the configuration message **500**, but the decoder **110** may not have undergone such an update. In other words, the encoder **108** and the decoder **110** may operate according to different versions of the syntax. In such examples, there may be a discrepancy in the format size of the payload part **520** as interpreted by the encoder **108** and as interpreted by the decoder **110**. For example, the format size according to the encoder **108**, being aware of the additional content of the payload part **520** and the updated payload format following the version update, is different from the format size according to the decoder **110**, since the decoder **110** is unaware of the additional content and the updated payload format and is operating according to the previous version. For the encoder **108**, the format size is equal to the actual size of the updated payload part **520**, but for the decoder **110** it is not. The format size alone may therefore be an unreliable indicator of the actual size of the payload part **520**. Including the payload size parameter **514** that specifies the actual size of the payload part **520** therefore facilitates more reliable processing of the decoder **110**.

[0138] In some examples, for example if the decoder **110** is in the first set of decoders, the first payload portion is parsed by the decoder **110** and the second payload portion is not parsed by the decoder **110**. Parsing the first payload portion but not the second payload portion enables the decoder **110** to obtain the first configuration data but not the second configuration data. In some examples, for example if the decoder **110** is in the second set of decoders, the second payload portion is parsed by the decoder **110** and the first payload portion is not parsed by the decoder **110**. Parsing the second payload portion but not the first payload portion **522** enables the decoder **110** to obtain the second configuration data but not the first configuration data. In some examples, for example if the decoder **110** is in the second set of decoders, both the first and the second payload portion are parsed to obtain both the first and the second configuration data.

[0139] As described above, the payload size parameter **514** may be used to determine the start of a following configuration message. In some examples, for example if the decoder **110** is in the first set of decoders, the content of the second payload portion is discarded. The content of the second payload portion may be discarded by using the total size of the payload part **520** specified in the payload size parameter **514**. Using the payload size parameter **514** to locate the start of the following message enables the decoder **110** to remain synchronised with the bytestream regardless of whether the decoder **110** is in the first set of decoders or the second set of decoders.

[0140] In a case where the decoder **110** obtains both the first and the second configuration data, the first representation **206** of the image is reconstructed using both the first and the second configuration data. In a case where the decoder **110** processes only one of the first and the second payload portions, the first representation **206** of the image may be reconstructed using the obtained one of the first and the second configuration data. As such, the use of the configuration message **500** to reconstruct the first representation **206** may be agnostic to the type of decoder and/or to the version number according to which the decoder **110** is configured to operate.

[0141] Referring to FIG. 7, there is shown a schematic diagram of an example of a configuration message **700**. The configuration message **700** may form part of configuration data **310**. The configuration message **700** may be generated by encoder **108**. Some items depicted in FIG. 7 are similar to items shown in FIG. 5. Corresponding reference signs, incremented by **200**, have therefore been used for similar items.

[0142] Configuration message **700** is a more specific example of configuration message **500**. Configuration message **700** comprises a header part **710** and a payload part **720**. The header part **710** comprises a message type parameter **712** and a payload size parameter **714**. The payload part **720** comprises a set of configuration parameters. The set of configuration parameters are related to processing of residual data **216**. In some examples, the configuration message **700** is an integer number of bytes long.

[0143] The message type of the configuration message **700** is an image set configuration message. An image set configuration message corresponds to multiple images in a set of images. In some examples, the set of images comprises **75** images. The set of images may be part of a video signal. For example, the set of images may comprise consecutive frames of a video. The video may be a streamed video. An example of a video is a broadcast video. For example, the video may be broadcast via the data communications network **106**. In some examples, the set of images is encoded as a group of pictures (GOP). A GOP is an encoding of a sequence of images each of which may be decoded using information contained entirely within the GOP. A video may comprise multiple such GOPs.

[0144] In some examples, an image set configuration message corresponds to every image in a set of images. The payload part **720** comprises a set of image set configuration parameters. The set of image set configuration parameters has a predetermined format.

[0145] In this example, the set of image set configuration parameters comprises a transform type parameter **751**. The transform type parameter **751** relates to a transformation operation performed on the residual data **216**. The transformation operation may be a directional decomposition operation. One example of a directional decomposition operation is a Hadamard transform. The transformation operation may exploit spatial and/or directional correlation between residual elements in the residual data **216**. Performing the transformation operation on the residual data **216** results in transformed residual data. The transformed residual data may be smaller than the residual data **216**. The transformed residual data may have a lower encoding entropy than the residual data **216**. As such, it may be more efficient for the encoder **108** to transmit the transformed residual data than the residual data **216**. The decoder **110**, receiving the transformed residual data, may then perform an inverse transformation operation on the transformed residual data to obtain the residual data **216**. The encoder **108** thus signals to the decoder **110** which transformation operation has been applied to the residual data **216** via the transform type parameter **751**. In some examples, the transform type parameter **751** specifies a transformation operation from a plurality of possible transformation operations that may be applied to the residual data **216**. In some examples, the value of the transform type parameter **751** has a fixed bit-length. For example, the value of the transform type parameter **751** may have a bit-length of two bits.

[0146] In this example, the set of image set configuration parameters comprises an upsample type parameter **752**. The upsample type parameter **752** relates to an upsampling operation. The upsampling operation is performed, for each image in the set of images, on a representation of the image at the relatively low level of quality. The upsampling operation is performed on the representation at the relatively low level of quality to derive the upsampled data **214**. The encoder **108** performs such an upsampling operation in order to derive the residual data **216**. The decoder **110** may perform a corresponding upsampling operation in order to reconstruct the input data **206**, namely by combining the upsampled data **214** with the residual data **216**. The encoder **108** thus signals to the decoder **110** which upsampling operation is to be used via the upsample type parameter **752**. In some examples, the upsample type parameter **752** specifies an interpolation operation from a plurality of possible interpolation operations. Examples of possible interpolation operations include nearest neighbour interpolation, bilinear interpolation, bicubic interpolation and Lanczos interpolation. In some examples, the upsample type parameter specifies whether upsampling is performed in one dimension or two dimensions. In other words, the upsample type parameter **752** may specify whether one or both of horizontal and vertical upsampling is performed

on the representation at the relatively low level of quality. In some examples, the configuration parameter relating to the upsampling operation may specify whether horizontal or both of horizontal and vertical upsampling is performed on the representation at the relatively low level of quality. In some examples, the value of the upsample type parameter **752** has a fixed bit-length. In one example, the value of the upsample type parameter **752** has a bit-length of three bits. In another example, the value of the upsample type parameter **752** has a bit-length of four bits.

[0147] In this example, the set of image set configuration parameters comprises a depth type parameter **753**. The depth type parameter **753** relates to a bit depth. The bit depth is associated with the residual data **216** for a given image in the set of images. The bit depth may also be referred to as a colour depth. In some examples, bit depth indicates the number of bits used to indicate the colour of a given pixel in a given image. As such, the bit depth may be indicative of how finely colour can be represented in the given image. In some examples, the bit depth indicates the number of bits used for each colour component of a given pixel in a given image. In some examples, a given residual element in the residual data **216** for a given image comprises a plurality of colour components (e.g. for YUV or RGB data). The bit depth may indicate the number of bits used for each colour component of the given residual element. Different colour components may have different bit lengths. In some examples, the value of the depth type parameter **753** has a fixed bit-length. In one example, the value of the depth type parameter **753** has a bit-length of two bits. In another example, the value of the depth type parameter **753** has a bit-length of four bits.

[0148] In this example, the set of image set configuration parameters comprises a temporal processing parameter **754**. The temporal processing parameter **754** specifies an extent of temporal processing associated with reconstructing, for a given image in the set of images, the input data **206**. Performing temporal processing comprises using data based on multiple images in the set of images. In some examples, each of the images in the set of images corresponds to a different time sample of a video. For example, temporal processing may adjust data for a given frame of video, such as data derived from residual data for the frame, using data for a different frame of video, such as data derived from residual data for the different frame. The data based on multiple images may be indicative of an extent of temporal correlation between residual data for the given image and residual data for one or more further images in the set of images. The one or more further images may correspond to one or more earlier time samples in the video relative to a time sample corresponding to the given image. For example, data at least derived from residual data that is associated with a previous frame of video may be used to reconstruct a current frame of video via modification of data at least derived from residual data that is associated with the current frame.

[0149] In some examples, the data based on multiple images comprises temporal correlation data. Temporal correlation data may comprise data that is based on a difference between residual data for the given image and corresponding residual data for the one or more further images. This may be a difference based on data for a current frame and data for a previous frame. For image sequences that exhibit a relatively high degree of temporal correlation, it may be more efficient to use and/or output the temporal correlation data than the residual data for the given image.

[0150] In some examples, the value of the temporal processing parameter **754** is based on an optimization metric. An optimization operation may be performed in order to determine whether the residual data or the temporal correlation data is more efficient to use and/or output for the given image. Based on the optimization metric, the value of the temporal processing parameter **754** may instruct the decoder **110** to reconstruct a given frame of video based on one of the residual data and the temporal correlation data. This may reflect a processing that has been performed at a corresponding encoder. The optimization operation may comprise a rate-distortion analysis, as described in more detail below.

[0151] The value of the temporal processing parameter **754** may have a bit-length of one bit. For example, this may indicate whether temporal processing is to be performed or not (a binary determination), e.g. in relation to a given frame of video. As such, the temporal processing

parameter **754** may be stored in a predetermined bit in a given byte of configuration data.

[0152] In this example, the set of image set configuration parameters comprises a resolution type parameter **755** specifying a resolution of the input data **206** for a given image in the set of images. In some examples, the value of the resolution type parameter **755** has a fixed bit-length. In one example, the value of the resolution type parameter **755** has a bit-length of seven bits.

[0153] In some examples, the set of image set configuration parameters comprises a custom resolution parameter **756**. The custom resolution parameter relates to a resolution of the input data **206**. The custom resolution parameter **756** may be included in the image set configuration message **700** selectively. This is depicted using dashed lines in FIG. 7. For example, the custom resolution parameter **756** may be included if it is determined that the resolution of the input data **206** is not one of a predetermined set of values that may be specified by the resolution type parameter **755**. Data may be stored in the resolution type parameter **755** to indicate that the custom resolution parameter **756** is included in the image set configuration message **700**. Such data may comprise a predetermined bit in the resolution type parameter **755**. The predetermined bit therefore comprises a parameter presence indicator that indicates the presence of the custom resolution parameter **756** in the image set configuration message **700**.

[0154] The set of image set configuration parameters may comprise more, fewer or different parameters than those described above with reference to FIG. 7.

[0155] The image set configuration message **700** corresponds to multiple images in the set of images. In this example, the image set configuration message **700** corresponds to all of the images in the set of images. As such, a single set of image set configuration parameters is applicable to every image in the set of images. Image set configuration data may also be referred to as “global configuration data”.

[0156] The image set configuration message **700** is output to the decoder **110** to enable the decoder **110** to reconstruct, for each image in the set of images, the input data **206** using the upsampled data **214** and the residual data **216**.

[0157] The number of times the image set configuration message **700** is output for the set of images is less than the number of images in the set of images. Outputting the image set configuration message **700** for a number of images that is less than the number of images in the set of images reduces the amount of information that is transmitted over a given period of time. In this example, the number of times the image set configuration message **700** is output for the set of images is one. The image set configuration message **700** may be output for the first image only in the set of images. The first image in the set of images may be the first image in the set of images that is output, received, processed and/or displayed. Outputting the image set configuration message **700** for the first image in the set of images may enable the recipient to receive the image set configuration data as early as possible, and/or may enable the recipient to apply the image set configuration data to as many images in the set of images as possible. In some examples, the image set configuration message **700** is output with or alongside the first image in the set of images. In some examples, the image set configuration message **700** is output for an image other than the first image in the set of images. In some examples, the number of times the image set configuration message **700** is output for the set of images is greater than one but less than the number of images in the set of images. In one example, the image set configuration message **700** is output once for a set of images comprising **75** images.

[0158] In some examples, the number of times the image set configuration message **700** is output for the set of images is based on an optimization metric. An example of an optimization metric is a rate-distortion metric. Rate-distortion optimization optimizes the amount of data that is transmitted against an amount of loss of visual quality. The more infrequently the image set configuration message **700** is output, the less data is transmitted. However, outputting the image set configuration message **700** too infrequently may result in an undesirable loss of quality due to the decoder **110** not being provided with up-to-date configuration data. Therefore, the number of times the image

set configuration message **700** is output is optimized so as to reduce the amount of data sent whilst reducing any loss in visual quality.

[0159] In some examples, a ratio of the number of times the image set configuration message **700** is output for the set of images to the number of images in the set of images is less than a predetermined threshold value. The predetermined threshold value may be based on an optimization metric such as the rate-distortion metric described above.

[0160] Referring to FIG. **8**, there is shown schematically an example of a configuration message **800**. The configuration message **800** may form part of configuration data **310**. The configuration message **800** may be generated by encoder **108**. Some items depicted in FIG. **8** are similar to items shown in FIG. **5**. Corresponding reference signs, incremented by 300, have therefore been used for similar items.

[0161] Configuration message **800** is a more specific example of configuration message **500**. The configuration message **800** comprises a header part **810** and a payload part **820**. The header part **810** comprises a message type parameter **812** and a payload size parameter **814**. The payload part **820** comprises a set of configuration parameters. The set of configuration parameters are related to processing of residual data **216**. In some examples, the configuration message **800** is an integer number of bytes long.

[0162] The message type of the configuration message **800** is an image configuration message. An image configuration message may correspond to a given image only in a set of images. As such, a different image configuration message may be generated and/or output for each image in the set of images. The payload part **820** comprises a set of image configuration parameters.

[0163] In this example, the set of image configuration parameters comprises a residual use parameter **851**. The value of the residual use parameter **851** specifies whether or not the input data **206** for a given image is to be reconstructed using the residual data **216** for the given image. If the input data **206** is to be reconstructed, a further set of configuration parameters is included in the image configuration message **800**. If the input data **206** is not to be reconstructed, the further set of configuration parameters is not included in the image configuration message **800**. Therefore, the further set of configuration parameters is included selectively. This is depicted using dashed lines in FIG. **8**.

[0164] In this example, the value of the residual use parameter **851** indicates whether the further set of configuration parameters is included in the image configuration message **800**. In this example, the value of the residual use parameter **851** can therefore indicate whether or not the further set of configuration parameters is included in the image configuration message **800**. As such, the residual use parameter **851** may comprise a parameter presence indicator. The residual use parameter **851** is useable by the decoder **110** to detect whether or not the input data **206** is to be reconstructed using the residual data **216** and whether or not the further set of configuration parameters are included in the configuration message **800**.

[0165] In some examples, when the further set of configuration parameters is included in the image configuration message **800**, the payload part **820** of configuration message **800** is 5 bytes long. When the further set of configuration parameters is not included in the configuration message **800**, the payload part **820** of configuration message **800** may be 1 byte long.

[0166] In this example, the set of image configuration parameters comprises quantisation data **852**. The quantisation data **852** is comprised in the further set of configuration parameters whose inclusion is indicated by the residual use parameter **851**. The quantisation data **852** is indicative of a quantisation level of data useable by the decoder **110** to reconstruct, for a given image, the input data **206** using the upsampled data **214**.

[0167] Quantisation involves approximating a relatively large set of values by a relatively small set of values, for example by rounding, thresholding or truncating the values in the relatively large set. The objective of a quantisation operation is to reduce the amount of data in quantised output data compared to that in the input data to the quantisation operation. A quantisation level of given data

may indicate how aggressively the given data is quantised. A quantisation level may be indicative of a granularity of quantised data obtained via the quantisation process. In another words, the quantisation level may be indicative of how much information is lost due to such quantisation. For example, a quantisation level may indicate one or more thresholds or bins for quantisation that indicate how data is quantised.

[0168] In some examples, the quantisation data **852** is indicative of a quantisation level of residual data **216**. Therefore, the data useable by the decoder **216** to reconstruct the input data **206** may be data at the relatively high level of quality. The quantisation level may instruct the decoder **110** on how to perform an inverse quantisation upon already quantised residual data **216**. For example, at the decoder **110**, a received set of residual data **216** may have values in a first quantised range, that are then expanded to a larger range of values following inverse quantisation. The quantisation data **852** may be used to perform this operation.

[0169] In some examples, the quantisation data **852** is indicative of a quantisation level of correction data. For a given image, the correction data is useable to correct for errors in encoding and/or decoding the representation at the relatively low level of quality, as described with reference to FIG. 2 above. Therefore, the data useable by the decoder **110** to reconstruct the input data **206** may be data at the relatively low level of quality.

[0170] In some examples, the quantisation data **852** is indicative of a first quantisation level and a second quantisation level. The first quantisation level is a quantisation level applied to the residual data **216**. The second quantisation level is a quantisation level applied to correction data. As such, both the residual data **216** and the correction data may be quantised using individually defined quantisation levels that are indicated by the quantisation data **852**. The quantisation data **852** may thus comprise levels at which one or more of correction data and residual data were quantised at an encoder (e.g. following transformation as described herein), which are then sent to a decoder to allow inverse quantisation and reconstruction of original values associated with the correction data and residual data (the reconstructed values may not match the original values prior to quantisation at the encoder, due to information lost during the quantisation process).

[0171] The quantisation data **852** may comprise an integer number of bytes. In some examples, the quantisation data **852** comprises two bytes. In some examples, the quantisation data **852** comprises four bytes.

[0172] In this example, the set of image configuration parameters comprises a picture type parameter **853**. The picture type parameter **853** is comprised in the further set of configuration parameters whose inclusion is indicated by the residual use parameter **851**. The picture type parameter **853** specifies whether the given image corresponds to a frame or a field. In other words, the picture type parameter **853** indicates whether the given image is progressive, in which case picture type parameter **853** specifies a frame, or interlaced, in which case picture type parameter **853** specifies a field. In a case where the given image is interlaced, the picture type parameter **853** may also specify a field type, for example a top field or a bottom field. The picture type parameter **853** may have a bit-length of one or two bits.

[0173] In this example, the set of image configuration parameters comprises a temporal refresh parameter **854**. The temporal refresh parameter **854** is comprised in the further set of configuration parameters whose inclusion is indicated by the residual use parameter **851**. The temporal refresh parameter **854** is used to indicate when temporal data is to be refreshed. For example, the value of the temporal refresh parameter **854** may specify whether or not a buffer storing temporal data is to be cleared. For example, as described herein, residual data (or transformed residual data) for a current frame of video may be modified based on residual data (or transformed residual data) for a previous frame of video. The latter data may be stored in a buffer and used to modify the former data. This may be repeated in an iterative manner for each frame. In one case, at an encoder, a current set of data and the contents of the buffer may be compared, and a difference computed. At the decoder, temporal processing may comprise adding the contents of the buffer to received data to

reconstruct residual or transformed residual data. In the present case, the value of the temporal refresh parameter **854** may indicate to the decoder that data relating to a previous frame that is stored in the buffer is to be cleared. This may effectively indicate that no temporal processing is to be applied (e.g. a value of **0** added to received data will not modify the received data). Temporal data comprises data useable in temporal processing, for example data that is based on multiple images. The bit-length of the value of the temporal refresh parameter **854** may be one bit.

[0174] The set of image configuration parameters may comprise more, fewer or different parameters than those described above with reference to FIG. **8**.

[0175] The image configuration message **800** is output for a plurality of images in the set of images. In some examples, the number of images in the plurality of images is equal to the number of images in the set of images. In other words, the image configuration message **800** may be output for every image in the set of images. In other examples, the number of images in the plurality of images is less than the number of images in the set of images. The number of images in the plurality of images may be based on an optimization metric. The optimization metric may be a rate-distortion metric.

[0176] Image set configuration message **700** and image configuration message **800** may both be output for a given set of images. The number of times image set configuration message **700** is output for the set of images is less than the number of times image configuration message **800** is output for the set of images. For example image set configuration message **700** may be output only once for the set of images, and image configuration message **800** may be output for every image in the set of images. As such, image configuration message **800** is output more frequently than image set configuration message **700**.

[0177] Outputting image configuration message **800** relatively frequently uses more data than outputting image configuration message **800** relatively infrequently. However, outputting image configuration message **800** relatively frequently provides the decoder **110** with up-to-date information that enables the decoder **110** to perform image reconstruction with a high degree of accuracy and/or reliability. The set of image configuration parameters stored in image configuration message **800** may change between different images, for example. There is thus a trade-off between an amount of data that is transmitted to the decoder **110** and an accuracy and/or reliability with which the decoder **110** can perform reconstruction using the data that is received. For image set configuration message **700** and image configuration message **800**, the optimal outcome of such a trade-off is that image configuration message **800** is sent more frequently than image set configuration message **700**. Sending image set configuration parameters with a relatively low frequency and sending image configuration parameters with a relatively high frequency may therefore reduce an amount of information that is sent over a given period of time whilst enabling the decoder **110** to perform reconstruction with a predetermined level of accuracy and/or reliability.

[0178] Referring to FIG. **9**, there is shown schematically an example of a configuration message **900**. The configuration message **900** may form part of configuration data **310**. The configuration message **900** may be generated by encoder **108**. Some items depicted in FIG. **9** are similar to items shown in FIG. **5**. Corresponding reference signs, incremented by **400**, have therefore been used for similar items.

[0179] Configuration message **900** is a more specific example of configuration message **500**. The configuration message **900** comprises a header part **910** and a payload part **920**. The header part **910** comprises a message type parameter **912** and a payload size parameter **914**. The payload part **920** comprises a set of configuration parameters. The set of configuration parameters are related to processing of residual data **216**. In some examples, the configuration message **900** is an integer number of bytes long.

[0180] The message type of the configuration message **900** is an encoded data message. An encoded data message may correspond to a given image in a set of images. As such, a different encoded data message may be generated and/or output for each image in the set of images.

[0181] The payload part **920** comprises a set of configuration parameters **950**. The payload part **920** may also comprise data **960** based on the residual data **216** for the given image. The data **960** based on the residual data **216** may comprise the residual data **216** itself. In some examples, the data **960** based on the residual data **216** comprises transformed residual data. Transformed residual data comprises data that is the output of a transformation operation performed on the residual data **216**. The transformation operation may be a directional decomposition operation that exploits correlation between residual elements in the residual data **216**.

[0182] In some examples, the data **960** based on the residual data **216** comprises encoded data. The encoded data may be encoded residual data or encoded transformed residual data. The encoded data may be derived using one or both of a run-length encoding operation and a Huffman encoding operation. As described above, the residual data or the transformed residual data may be quantised prior to encoding.

[0183] In some examples, the data **960** based on the residual data **216** is stored in one or more variable-length elements each comprising an integer number of bytes, as described above.

[0184] In this example, the set of configuration parameters **950** indicates whether the encoded data is derived using a Huffman encoding operation, a run-length encoding operation, or both. In some examples, the set of configuration parameters **950** indicates whether or not the encoded data is output for the given image. In some examples, the set of configuration parameters **950** relates to a size of the encoded data. For example, a configuration parameter may relate to a size of run-length encoded data and/or a size of Huffman encoded data.

[0185] The encoded data message **900** is output for a plurality of images in a set of images. In some examples, the number of images in the plurality of images is equal to the number of images in the set of images. In other words, the encoded data message **900** may be output for every image in the set of images. In other examples, the number of images in the plurality of images is less than the number of images in the set of images. The number of images in the plurality of images may be based on an optimization metric. The optimization metric may be a rate-distortion metric.

[0186] In some examples, the encoded data message **900** is generated and/or output selectively. Whether or not an encoded data message is generated and/or output for a given image may depend on whether the input data **206** for the given image is to be reconstructed using the residual data **216** for the given image. Whether or not the input data **206** for the given image is to be reconstructed may be indicated by the residual use parameter **851** of the corresponding image configuration message **800** for the given image. An amount of information that is transmitted may be reduced by outputting the encoded data message **900** selectively compared to a case in which the encoded data message **900** is not selectively output.

[0187] Configuration messages generated by the encoder **108** may have other message types than those described above. Another example message type is an encoder metadata message. An encoder metadata message comprises metadata relating to one or more encoder characteristics. Another example message type is a decoder control message. A decoder control message comprises instructions for controlling a decoder. A decoder control message may comprise a configuration parameter relating to image dithering. Another example message type is a research message. A research message comprises a custom payload for research purposes.

[0188] Referring to FIG. **10**, there is shown an example of a method **1000** of processing configuration data. The method **1000** may be performed by an apparatus comprising an encoder such as the first apparatus **102** described above.

[0189] At item **1010**, for each image in a set of images, a representation of the image at the relatively low level of quality is derived using a first representation of the image at the relatively high level of quality. The representation at the relatively low level of quality is output for processing by the decoder **110**.

[0190] At item **1020**, residual data is output for one or more selected images in the set of images. The residual data is useable to reconstruct the first representation using a second representation of

the image at the relatively high level of quality, the second representation being based on the representation of the image at a relatively low level of quality. The residual data is derived based on the first representation and the second representation.

[0191] In some examples, the one or more selected images comprises all of the images in the set of images. In some examples, the one or more selected images are selected based on image content. For example, if the first representation of a first image contains a relatively large amount of visual detail, it may be determined that residual data for the first image is to be used and/or output to enable the first representation of the first image to be reconstructed. Contrastingly, if the first representation of a second image contains relatively little visual detail, for example if the second image corresponds to a “black screen”, it may be determined that residual data for the second image is not to be used and/or output. Using residual data to reconstruct the first representation of the second image may not substantially improve the displayed visual quality for the second image, due to the lack of visual detail contained therein. Using and/or outputting residual data selectively may reduce an amount of information that is transmitted and/or processed over a given period of time compared with not selectively using and/or outputting residual data.

[0192] At item **1030**, configuration data is output for processing by the decoder **110**. The configuration data enables the decoder **110** to detect whether the first representation of a given image in the set of images is to be reconstructed using residual data for the given image. The given image may be comprised in the one or more selected images. The configuration data may be output for each image in the set of images. The configuration data may comprise image configuration message **800**. The configuration data may, for example comprise the residual use parameter **851**. In some examples, the configuration data comprises encoded data message **900**.

[0193] In some examples, the number of times the configuration data is output for the set of images is less than the number of images in the set of images. The configuration data may correspond to multiple images in the set of images. The configuration data may comprise image set configuration message **700**, for example. In some examples, the number of times the configuration data is output for the set of images is based on an optimization metric.

[0194] Referring to FIG. **11**, there is shown an example of a method **1100** for use in processing representations of an image. The method **1100** may be performed by the decoder **110** described above.

[0195] At item **1110**, data is received that is useable to derive a representation of an image at the relatively low level of quality. Prior to receipt by the decoder **110**, the representation at the relatively low level of quality is derived by encoder **108** using a first representation of the image at the relatively high level of quality. In some examples, the image is one of a set of images. The set of images may be part of a video signal. In some examples, the received data is processed to obtain the representation at the relatively low level of quality. For example, the received data may be decoded to obtain the representation at the relatively low level of quality.

[0196] At item **1120**, it is determined whether predetermined configuration data has been received. The predetermined configuration data corresponds to the image.

[0197] If it is determined at item **1120** that the predetermined configuration data corresponding to the image has been received, the first representation of the image is reconstructed at item **1130**. The first representation is reconstructed using the predetermined configuration data. The first representation is reconstructed using a second representation of the image at the relatively high level of quality. In some examples, the representation at the relatively low level of quality is processed to derive the second representation. For example, the second representation may be derived by upsampling the representation at the relatively low level of quality. The first representation may be reconstructed by combining the second representation with the residual data.

[0198] At item **1135**, the first representation is output. For example, the first representation may be output for display by a display device.

[0199] If it is determined at item **1120** that the predetermined configuration data corresponding to

the image has not been received, the first representation is not reconstructed. Instead, the representation at the relatively low level of quality is processed at item **1140** to derive the second representation. The second representation may be derived by upsampling the representation at the relatively low level of quality.

[0200] At item **1145**, the second representation is output. For example, the second representation may be output for display by the display device.

[0201] In some examples, if it is determined that the predetermined configuration data corresponding to the image has not been received, the second representation is not derived. For example, the representation at the relatively low level of quality may be output instead of the second representation. In some examples, data based on the representation at the relatively low level of quality is output. The data based on the representation at the relatively low level of quality is different from the first representation and the second representation.

[0202] In a case where the image is one of a set of images, the predetermined configuration data may correspond to multiple images in the set of images. The predetermined configuration data may not be received with every image in the set of images. In one example, the predetermined configuration data is received with the first image only in the set of images. The predetermined configuration data may be applicable to every subsequent image in the set of images. As such, if the first image in the set of images is not received by the decoder **110**, the predetermined configuration data for the set of images may also not be received. The predetermined configuration data may be image set configuration data. The predetermined configuration data may comprise image set configuration message **700**.

[0203] In a case where the set of images are part of a streamed video, the decoder may enter the video stream partway through the set of images. In such a case, although the first representation is not reconstructed due to the absence of receipt of the predetermined configuration data, a representation at the relatively high level of quality, namely the second representation, may still be output. Outputting the second representation for display reduces a potential loss in visual quality caused by the absence of receipt of the predetermined configuration data compared to a case in which a representation at the relatively high level of quality is not output. Outputting the second representation may be considered part of a “minimum operation mode”, in contrast to outputting the first representation, which may be considered part of a “full operation mode”. In the case where the decoder **110** enters the video stream partway through the set of images, the “minimum operation mode” may be used for the remainder of the set of images. When the first image in a following set of images is then received, the predetermined configuration data for the following set of images is also received. As such, the following set of images may be processed according to the “full operation mode”.

[0204] Referring to FIG. **12**, there is shown schematically an example of a data processing technique **1200**. Some or all of the data processing technique **1200** may be performed by an apparatus comprising an encoder such as the first apparatus **102** described above and/or by an apparatus comprising a decoder such as the second apparatus **104**.

[0205] The first Low Complexity Enhancement Video Coding (LCEVC) standard is now published as ISO/IEC 23094-2, published October 2021.

[0206] Examples that will now be described add extensions to the first LCEVC standard to provide a new LCEVC standard, which may be referred to herein as an “extended” LCEVC standard.

[0207] The extended LCEVC standard aims to maintain the low complexity of the first LCEVC standard.

[0208] The extended LCEVC standard is configured such that any extension to an LCEVC bitstream of the first LCEVC standard maintains backwards compatibility with the first LCEVC standard. For example, an LCEVC decoder designed to decode an LCEVC bitstream in accordance with the first LCEVC standard (which may be referred to hereinafter as a “legacy”, “unextended”, or “old” decoder), should be able to decode, at least to some extent (for example, without crashing),

a bitstream generated according to the extended LCEVC standard (which may be referred to as an “extended” bitstream). In other words, a legacy LCEVC decoder that is configured to decode the first LCEVC standard should also be able to process a bitstream generated according to the extended LCEVC standard and output data suitable for displaying on a screen. The output data may not be exactly the same as what might be output by an “extended” LCEVC decoder (which may also be referred to as an “updated” (LCEVC) decoder), i.e. one that is configured to decode the extended LCEVC standard. However, the output data is at least data suitable for displaying an image. A less effective outcome is that, in such a scenario, the legacy LCEVC decoder configured to decode the first LCEVC standard crashes upon receipt and/or processing of the extended bitstream generated according to the extended standard. In such a situation, a user of such a device would not be able to view any video at all.

[0209] Examples described herein address these matters.

[0210] Although examples described herein refer to the first LCEVC standard and an extended LCEVC standard, it will be understood that they can equally be applied to any different (e.g. future) versions of the LCEVC standard.

[0211] The below-described examples use the general principle that LCEVC (data) blocks are independent of each other. For example, this includes signalling blocks (e.g. configuration blocks). The term “block” is generally used herein to mean a group of one or more bytes with semantics associated with the byte(s). A block may comprise an integer number of bytes.

[0212] In accordance with examples, an (e.g. LCEVC) encoder is provided, which is configured to generate a configuration block **1210** (i.e. a block of configuration data).

[0213] In some examples, the configuration block **1210** comprises global configuration data. The configuration block **1210** instructs a decoder to use a given configuration. The configuration block **1210** signals to a decoder to select the given configuration out of multiple possible configurations. In examples, the configuration block **1210** signals the selection via a configuration value **1220**. The encoder may generate one or more configuration blocks **1210**. Each of the one or more configuration blocks **1210** may comprise one or more configuration values **1220**.

[0214] By way of a specific and non-limiting example, the configuration block **1210** may comprise a one-byte block. The configuration block **1210** may comprise a one-bit ‘temporal R sign’ parameter, a one-bit ‘temporal enabled’ parameter, a three-bit ‘upsample’ parameter, a one-bit ‘filter’ parameter, and a 2-bit ‘sml’ parameter. In this example, the three-bit upsample parameter may therefore take one of eight different values, some or all of which may indicate a different upsample type. Again, by way of a specific and non-limiting example, an upsample parameter value ‘0’ may indicate ‘nearest’ upsample type, ‘1’ may indicate ‘linear’ upsample type, ‘2’ may indicate ‘cubic’ upsample type, ‘3’ may indicate ‘modified cubic’ upsample type, ‘4’ may indicate ‘adaptive cubic’ type, ‘5’ may indicate a first reserved value, ‘6’ may indicate a second reserved value, and ‘7’ may indicate an unspecified upsample type.

[0215] In principle, the types of upsampler associated with each upsample parameter value could be updated over time. For example, upsample parameter value ‘5’ could be changed from being reserved to being a specific type of upsampler in the extended LCEVC standard. While an extended legacy decoder would then be able to use that specific type of upsampler, a legacy decoder would interpret an upsample parameter value ‘5’ as being a reserved value and would likely crash.

[0216] As such, in examples, the encoder is also configured to generate a new extension configuration block **1240** (i.e. a block of extension configuration data). In some examples, the extension configuration block **1240** comprises global configuration data. The extension configuration block **1240**, when processed by a decoder, indicates one or more predetermined actions to be performed in relation to the selection of the configuration indicated by the configuration block **1210**. In examples, the extension configuration block **1240** signals the action(s) via an extension configuration value **1250**. The encoder may generate one or more extension configuration blocks **1240**. Each of the one or more extension configuration blocks **1240** may

comprise one or more extension configuration values **1250**.

[0217] By way of a specific and non-limiting example, the extended configuration block **1240** may comprise a one-byte block. The extended configuration block **1240** may also comprise a three-bit upsample parameter, in addition to one or more additional parameters. In this example, the three-bit upsample parameter may therefore take one of eight different values. In some examples, each of the three-bit upsample parameters indicates a different upsample type. However, as will be explained in more detail below, in other examples, at least one of the upsample parameters does not indicate an upsample type as such.

[0218] A configuration block **1210** may be used to signal (i.e. to a decoder) a selection of a specific upsampler. In particular, a value **1220** of a configuration block **1210** may signal (i.e. to a decoder) a selection of a first (specific) upsampler. For example, a value **1220** of the configuration block **1210** being equal to '1' may cause a decoder to select a linear upsampler. A value **1220** of the configuration block **1210** being equal to '2' may cause a decoder to select a cubic upsampler.

[0219] An extension configuration block **1240** may be used to perform one or more predetermined actions in relation to the configuration signalled by the configuration block **1210**. Examples of such predetermined actions include, but are not limited to, update, modify, adjust, use, overwrite and ignore the configuration signalled by the configuration block **1210**. The term "update" as used herein does not imply that the configuration is modified; updating the configuration may result in the same or a different configuration.

[0220] For example, the encoder may be configured to generate a configuration block **1210** having a value **1220** equal to '1' and an extended configuration block **1240** having a value **1250** equal to '2'. In this example, a suitably configured (i.e. updated) decoder may update the configuration value **1220** (i.e. the value **1220** '1' of the configuration block **1210**) by the extension configuration value **1250** (i.e. the value **1250** '2' of the extension configuration block **1240**) to produce an updated configuration value. For example, the decoder may increment the configuration value **1220** '1' by the extension configuration value **1250** '2', to arrive at an updated configuration value of '3' and a corresponding updated configuration. On the other hand, a decoder that has not been configured to process the extension configuration block **1240** ignores the extension configuration block **1240** and continues to decode according to the configuration value **1220** (i.e. '1'). In some examples, the extension configuration value **1250** is selected such that the updated configuration value falls outside the range of values in the configuration block **1210**, indicating that a value in the extension configuration block **1240** should be used. For example, with a configuration value **1220** of '1' and an extension configuration value **1250** of '7', the updated configuration value would fall outside the range of values ('0' to '7') in the configuration block **1210** and the appropriate value (i.e. '1') in the extension configuration block **1240** would be used.

[0221] In another example, where the extended configuration block **1240** has a value **1250** equal to '0' (an example of a first predetermined extension configuration value), a suitably configured (i.e. updated) decoder uses the configuration value **1220** of the configuration block **1210** (rather than a different value **1250** in the extended configuration block **1240**) to select the upsample type. As such, the decoder, in effect, ignores the upsample types in the extended configuration block **1240** and uses an upsample type from the configuration block **1210**. However, where the extended configuration block **1240** has a non-zero value **1250**, i.e. a value **1250** not equal to '0', (an example of a second predetermined extension configuration value), a suitably configured (i.e. updated) decoder uses the extended configuration value **1250** of the extended configuration block **1240**. As such, the decoder, in effect, ignores the upsample types in the configuration block **1210** and uses an upsample type from the extended configuration block **1240**.

[0222] In some examples, a suitably configured (i.e. updated) decoder determines whether the extended configuration value **1250** has one or more first predetermined extension configuration values (in this example, only '0'). If the decoder determines that the extended configuration value **1250** does not have the one or more first predetermined extension configuration values (in this

example, only '0')-in other words, the extended configuration value **1250** has one or more second predetermined extension configuration values (in this example, '1', '2', '3', '4', '5', '6' or '7'), the decoder does not determine the upsampler type parameter value of the configuration block **1210** as it will not be used. In other examples, the decoder may still determine the upsampler type parameter value of the configuration block **1210**.

[0223] In some examples, the configuration block **1210** and the extension configuration block **1240** relate to (i.e. signal to a decoder) a type of upsampler.

[0224] In such examples, the configuration value **1220** of '1' may signal to a decoder to use a linear upsampler. If an extended decoder then adjusts this value to '3', as per the above example, the decoder may proceed to decode the data using a different, e.g modified cubic upsampler.

[0225] In this example, it can thus be seen that a legacy decoder can still decode the data. This is because the legacy decoder proceeds with the configuration value **1220** of '1', as signalled by the configuration block **1210**. However, the legacy decoder decodes the data using a one upsampler type whereas a decoder according to examples (i.e. an extended encoder) potentially decodes the data using another upsampler type. The decoding is only 'potentially' using another upsampler type because, in some examples, an extended decoder may still use the same upsampler type as a legacy decoder.

[0226] In accordance with examples, a bitstream comprising a configuration block **1210** is provided. The bitstream may comprise an extension configuration block **1240**. In general, the configuration block **1210** and the extension configuration block **1240** are associated with a common (or "shared") configuration. The extension configuration block **1240** may be configured to adjust and/or modify a configuration as signalled by the configuration block **1210**.

[0227] A legacy decoder (e.g. a decoder configured to decode according to the first LCEVC standard) is configured to process the configuration block **1210**. In examples, the legacy decoder is configured to process the configuration block **1210** to determine a configuration, for example a value **1220** and/or setting and/or selection for a configuration. The unextended decoder ignores the extension block **1240**. The unextended decoder thus decodes the bitstream in accordance with the configuration block **1210**. The decoding as performed by the legacy decoder is not affected by the presence of the extension configuration block **1250** in the received bitstream. The bitstream may be an LCEVC compliant bitstream.

[0228] As shown in FIG. 12, in examples, the extension configuration block **1240** is separate from the configuration block **1210** in the bitstream. In this specific example, the extension configuration block **1240** follows the configuration block **1210** and all other configuration blocks **1210** in the bitstream, i.e. the extension configuration block **1240** is at the end of the bitstream. In this way, a legacy decoder can stop reading configuration at the end of the configuration block(s) **1210**, thereby ignoring the extension configuration block(s) **1240**. However, the extension configuration block **1240** may be in a different position in the bitstream in other examples.

[0229] In accordance with examples, an extended decoder configured to process the configuration block **1210** is provided. In examples, the extended decoder is configured to process the configuration block **1210** to determine a configuration. The configuration block **1210** is associated with the configuration. The extended decoder is configured to process the extension configuration block **1240**. By processing the extension configuration block **1240**, the extended decoder updates the configuration (indicated by the configuration value **1220**) or performed another predetermined action to determine an "extension" (or "extended") configuration. The extended decoder is configured to decode in accordance with the extension configuration. In some examples, the extended decoder processes the extension configuration block **1240** to override and/or overrun the configuration of the configuration block **1210**.

[0230] In examples, an extended decoder may, in response to processing the extension configuration block **1240**, increment a configuration value **1220** associated with the configuration block **1210**. In examples, the incrementation is indicated by an extension configuration value **1250**

of the extension configuration block **1240**.

[0231] In some examples in which incrementation is used, the extension configuration value **1250** may be selected such that the incrementing does not result in reserved values being derived, since such could result in the decoder crashing.

[0232] As such, examples can be used for configurations and/or tools that have ‘fields’ and/or ‘options’ that are already in the bitstream. This may be, for example, a selection of a type of upsampler to be used, a selection of a temporal setting, and/or a selection of a filter.

[0233] Thus, examples may be used to provide new selections of such configurations and/or settings, without crashing any legacy decoders. In preferred examples, the encoders or decoders are part of a tier-based hierarchical coding scheme or format. Examples of a tier-based hierarchical coding scheme include LCEVC: MPEG-5 Part 2 LCEVC (“Low Complexity Enhancement Video Coding”) and VC-6: SMPTE VC-6 ST-2117, the former being described in PCT/GB2020/050695, published as WO 2020/188273, (and the associated standard document) and the latter being described in PCT/GB2018/053552, published as WO 2019/111010, (and the associated standard document), all of which are incorporated by reference herein. However, the concepts illustrated herein need not be limited to these specific hierarchical coding schemes. A further example is described in WO2018/046940, which is incorporated by reference herein.

[0234] LCEVC (Low-Complexity Enhancement Video Coding) is a standardised coding method set out in standard specification documents including the Text of ISO/IEC 23094-2 Ed 1 Low Complexity Enhancement Video Coding published in October 2021, which is incorporated by reference herein. LCEVC can be used to efficiently encode and decode an image, further implementation details for these types of encoders and decoders are set out in earlier-published patent applications GB1615265.4 and WO2020188273, each of which is incorporated here by reference.

[0235] Further implementation details, for example of the LCEVC bitstream in published patent applications WO2020/089617, WO2020/089615, WO2020/089616, WO2020/089614 each of which is incorporated here by reference.

[0236] Referring to FIG. **13**, there is shown a schematic block diagram of an example of an apparatus **1300**.

[0237] In an example, the apparatus **1300** comprises an encoder. In another example, the apparatus **1300** comprises a decoder.

[0238] Examples of apparatus **1300** include, but are not limited to, a mobile computer, a personal computer system, a wireless device, base station, phone device, desktop computer, laptop, notebook, netbook computer, mainframe computer system, handheld computer, workstation, network computer, application server, storage device, a consumer electronics device such as a camera, camcorder, mobile device, video game console, handheld video game device, or in general any type of computing or electronic device.

[0239] In this example, the apparatus **1300** comprises one or more processors **1301** configured to process information and/or instructions. The one or more processors **1301** may comprise a central processing unit (CPU). The one or more processors **1301** are coupled with a bus **1302**. Operations performed by the one or more processors **1301** may be carried out by hardware and/or software. The one or more processors **1301** may comprise multiple co-located processors or multiple disparately located processors.

[0240] In this example, the apparatus **1300** comprises computer-useable volatile memory **1303** configured to store information and/or instructions for the one or more processors **1301**. The computer-useable volatile memory **1303** is coupled with the bus **1302**. The computer-useable volatile memory **1303** may comprise random access memory (RAM).

[0241] In this example, the apparatus **1300** comprises computer-useable non-volatile memory **1304** configured to store information and/or instructions for the one or more processors **1301**. The computer-useable non-volatile memory **1304** is coupled with the bus **1302**. The computer-useable

non-volatile memory **1304** may comprise read-only memory (ROM).

[0242] In this example, the apparatus **1300** comprises one or more data-storage units **1305** configured to store information and/or instructions. The one or more data-storage units **1305** are coupled with the bus **1302**. The one or more data-storage units **1305** may for example comprise a magnetic or optical disk and disk drive or a solid-state drive (SSD).

[0243] In this example, the apparatus **1300** comprises one or more input/output (I/O) devices **1306** configured to communicate information to and/or from the one or more processors **1301**. The one or more I/O devices **1306** are coupled with the bus **1302**. The one or more I/O devices **1306** may comprise at least one network interface. The at least one network interface may enable the apparatus **1300** to communicate via one or more data communications networks. Examples of data communications networks include, but are not limited to, the Internet and a Local Area Network (LAN). The one or more I/O devices **1306** may enable a user to provide input to the apparatus **1300** via one or more input devices (not shown). The one or more input devices may include for example a remote control, one or more physical buttons etc. The one or more I/O devices **1306** may enable information to be provided to a user via one or more output devices (not shown). The one or more output devices may for example include a display screen.

[0244] Various other entities are depicted for the apparatus **1300**. For example, when present, an operating system **1307**, data signal processing module **1308**, one or more further modules **1309**, and data **1310** are shown as residing in one, or a combination, of the computer-usable volatile memory **1303**, computer-usable non-volatile memory **1304** and the one or more data-storage units **1305**. The data signal processing module **1308** may be implemented by way of computer program code stored in memory locations within the computer-usable non-volatile memory **1304**, computer-readable storage media within the one or more data-storage units **1305** and/or other tangible computer-readable storage media. Examples of tangible computer-readable storage media include, but are not limited to, an optical medium (e.g., CD-ROM, DVD-ROM or Blu-ray), flash memory card, floppy or hard disk or any other medium capable of storing computer-readable instructions such as firmware or microcode in at least one ROM or RAM or Programmable ROM (PROM) chips or as an Application Specific Integrated Circuit (ASIC).

[0245] The apparatus **1300** may therefore comprise a data signal processing module **1308** which can be executed by the one or more processors **1301**. The data signal processing module **1308** can be configured to include instructions to implement at least some of the operations described herein. During operation, the one or more processors **1301** launch, run, execute, interpret or otherwise perform the instructions in the signal processing module **1308**.

[0246] Although at least some aspects of the examples described herein with reference to the drawings comprise computer processes performed in processing systems or processors, examples described herein also extend to computer programs, for example computer programs on or in a carrier, adapted for putting the examples into practice. The carrier may be any entity or device capable of carrying the program.

[0247] It will be appreciated that the apparatus **1300** may comprise more, fewer and/or different components from those depicted in FIG. **13**.

[0248] The apparatus **1300** may be located in a single location or may be distributed in multiple locations. Such locations may be local or remote.

[0249] The techniques described herein may be implemented in software or hardware, or may be implemented using a combination of software and hardware. They may include configuring an apparatus to carry out and/or support any or all of techniques described herein.

[0250] Examples described herein relate to a syntax for the use of configuration data relating to processing of residual data, the residual data being useable in image reconstruction. The syntax provides a processing framework to enable an encoder to obtain and output configuration data and to enable a decoder to receive and process the configuration data. The syntax framework facilitates the generation and processing of configuration messages such as those described above. A decoder

uses received configuration data to reconstruct representations of images, for example that are part of a video stream, at a high level of quality. The syntax framework described herein allows the decoder to perform such reconstruction with a relatively high degree of efficiency, reliability and accuracy.

[0251] The syntax framework described herein enables an overhead cost of transmitted data to be reduced compared to some known systems. The syntax framework described herein may involve an overhead of at most **3** bytes for a given image when residual data is not to be used to reconstruct the representation of the given image at the high level of quality. When residual data is to be used to reconstruct the representation at the high level of quality, an overhead cost of at most **43** bytes for the given image may be involved. This is in contrast to some known systems, which may involve an overhead cost per image of between **108** and **188** bytes. As such, an amount of information that is generated, stored, processed and/or transmitted over a given period of time is significantly reduced through the use of the syntax framework described herein.

[0252] It is to be understood that any feature described in relation to any one embodiment may be used alone, or in combination with other features described, and may also be used in combination with one or more features of any other of the embodiments, or any combination of any other of the embodiments. Furthermore, equivalents and modifications not described above may also be employed without departing from the scope of the invention, which is defined in the accompanying claims.

[0253] Further features are set out in the following numbered clauses:

[0254] 1. A method comprising: [0255] deriving, for each image in a set of images, a representation of the image at a first level of quality in a tiered hierarchy having multiple levels of quality using a first representation of the image at a second, higher level of quality in the tiered hierarchy; [0256] outputting, for each image in the set of images, the representation of the image at the first level of quality for processing by a decoder; and [0257] outputting configuration data for processing by the decoder to enable the decoder to detect whether or not the first representation of a given image in the set of images is to be reconstructed using residual data for the given image, the residual data being useable to reconstruct the first representation using a second representation of the image at the second level of quality, the second representation being based on the representation of the image at the first level of quality, the residual data being derived based on the first representation and the second representation.

[0258] 2. A method according to clause 1, the method comprising outputting, for one or more selected images in the set of images, the residual data.

[0259] 3. A method according to clause 1 or clause 2, wherein the configuration data is output for each image in the set of images.

[0260] 4. A method according to clause 1 or clause 2, wherein the number of times the configuration data is output for the set of images is less than the number of images in the set of images.

[0261] 5. A method according to any of clauses 1 to 4, wherein the one or more selected images comprises all of the images in the set of images.

[0262] 6. A method according to any of clauses 1 to 5, wherein the configuration data corresponds to multiple images in the set of images.

[0263] 7. A method according to any of clauses 1 to 6, the method comprising performing byte-wise processing of the configuration data.

[0264] 8. A method according to any of clauses 1 to 7, wherein, in a case where the configuration data indicates that the first representation of the given image is not to be reconstructed using the residual data for the given image, the configuration data is at most three bytes long.

[0265] 9. A method according to any of clauses 1 to 8, wherein, in a case where the configuration data indicates that the first representation of the given image is to be reconstructed using the residual data for the given image, the configuration data is at most forty-three bytes long.

[0266] 10. A method comprising: [0267] obtaining, for each image in a set of images, a representation of the image at a first level of quality in a tiered hierarchy having multiple levels of quality, the representation of the image at the first level of quality having been derived by an encoder using a first representation of the image at a second, higher level of quality in the tiered hierarchy; [0268] detecting, based on received configuration data, whether the first representation of a given image in the set of images is to be reconstructed using residual data for the given image, the residual data being useable to reconstruct the first representation using a second representation of the image at the second level of quality, the second representation being based on the representation of the image at the first level of quality, the residual data being derived based on the first representation and the second representation; and [0269] processing, for each image in the set of images, the representation of the image at the first level of quality based on the received configuration data.

[0270] 11. A method according to clause 10, the method comprising receiving, for one or more selected images, the residual data.

[0271] 12. A method according to clause 10, the method comprising, in response to detecting that the first representation of the given image is not to be reconstructed using the residual data for the given image: [0272] processing the representation of the given image at the first level of quality by upsampling the representation of the given image at the first level of quality to obtain the second representation of the given image at the second level of quality; and [0273] outputting the second representation of the given image.

[0274] 13. A method according to any of clauses 10 to 12, the method comprising, in response to detecting that the first representation of the given image is to be reconstructed using the residual data for the given image: [0275] processing the representation of the given image at the first level of quality by upsampling the representation of the given image at the first level of quality to obtain the second representation of the given image at the second level of quality; [0276] using the residual data for the given image and the second representation of the given image to obtain the first representation of the given image; and [0277] outputting the first representation of the given image.

[0278] 14. A method according to any of clauses 10 to 13, the method comprising receiving the configuration data for each image in the set of images.

[0279] 15. A method according to any of clauses 10 to 14, wherein the configuration data corresponds to multiple images in the set of images.

[0280] 16. A method according to any of clauses 10 to 15, wherein the one or more selected images comprises all of the images in the set of images.

[0281] 17. A method according to any of clauses 10 to 16, wherein the given image is comprised in the one or more selected images.

[0282] 18. A method of processing configuration data, the method comprising: [0283] obtaining residual data based on a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality and a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy, the residual data being useable by a decoder to reconstruct the first representation using the second representation; [0284] generating configuration data relating to processing of the residual data, the configuration data being useable by the decoder to reconstruct the first representation, wherein the configuration data comprises a parameter presence indicator that specifies whether or not a given set of configuration parameters are comprised in a given part of the configuration data, wherein the given set of configuration parameters has a predetermined format; and [0285] outputting the configuration data for processing by the decoder to enable the decoder to reconstruct the first representation using the second representation and the residual data.

[0286] 19. A method according to clause 18, wherein the given set of configuration parameters

comprises a configuration parameter having a variable bit-length and that is stored in a variable-length element, the variable-length element comprising at least one given byte having one or more predetermined bits arranged to indicate whether or not the variable-length element comprises one or more additional bytes to the at least one given byte.

[0287] 20. A method according to clause 18 or clause 19, wherein the given set of configuration parameters comprises a configuration parameter relating to a resolution of the first representation of the image.

[0288] 21. A method according to clause 18 or clause 19, wherein the parameter presence indicator is comprised in a residual use parameter, wherein the residual use parameter is useable by the decoder to detect whether or not the first representation of the image is to be reconstructed using the residual data.

[0289] 22. A method according to any of clauses 18 to 21, wherein the given set of configuration parameters comprises quantisation data indicative of a quantisation level of data useable by the decoder to reconstruct the first representation of the image.

[0290] 23. A method according to any of clauses 18 to 22, wherein the given set of configuration parameters comprises an image type parameter that specifies whether the first representation of the image relates to a progressive or an interlaced video.

[0291] 24 A method according to any of clauses 18 to 23, wherein in a case where the first representation of the image relates to an interlaced video, the image type parameter also specifies a field type.

[0292] 25. A method according to any of clauses 18 to 24, wherein the given set of configuration parameters comprises a version number according to which the decoder is configured to operate.

[0293] 26. A method according to any of clauses 18 to 25, wherein the parameter presence indicator comprises at least one predetermined bit in a predetermined byte of the configuration data.

[0294] 27. A method according to any of clauses 18 to 26, the method comprising performing byte-wise processing on the configuration data.

[0295] 28. A method of processing configuration data, the method comprising: [0296] receiving configuration data relating to processing of residual data, the residual data having been obtained, for a given image in a set of images, based on a first representation of the image at a first level of quality in a tiered hierarchy having multiple levels of quality and a second representation of the image at the first level of quality, the second representation being based on a representation of the given image at a second, lower level of quality in the tiered hierarchy, the residual data being useable to reconstruct the first representation using the second representation; and [0297] processing the configuration data to reconstruct, for the given image, the first representation using the second representation and the residual data, [0298] wherein the configuration data comprises a parameter presence indicator that specifies whether or not a given set of configuration parameters are comprised in a given part of the configuration data, the given set of configuration parameters having a predetermined format.

[0299] 29. A method according to clause 28, wherein the given set of configuration parameters comprises a configuration parameter having a variable bit-length and that is stored in a variable-length element, the variable-length element comprising at least one given byte having one or more predetermined bits arranged to indicate whether or not the variable-length element comprises one or more additional bytes to the at least one given byte.

[0300] 30. A method according to clause 28 or clause 29, wherein the given set of configuration parameters comprises a configuration parameter relating to a resolution of the first representation of the image.

[0301] 31. A method according to clause 28 or clause 29, wherein the parameter presence indicator is comprised in a residual use parameter, wherein the residual use parameter is useable to detect whether or not the first representation of the image is to be reconstructed using the residual data.

[0302] 32. A method according to any of clauses 28 to 31, wherein the given set of configuration

parameters comprises quantisation data indicative of a quantisation level of data useable by the decoder to reconstruct the first representation of the image.

[0303] 33. A method according to any of clauses 28 to 32, wherein the given set of configuration parameters comprises an image type parameter that specifies whether the first representation of the image relates to a progressive or an interlaced video.

[0304] 34. A method according to any of clauses 28 to 33, wherein in a case where the first representation of the image relates to an interlaced video, the image type parameter also specifies a field type.

[0305] 35. A method according to any of clauses 28 to 34, wherein the given set of configuration parameters comprises a version number.

[0306] 36. A method according to any of clauses 28 to 35, wherein the parameter presence indicator comprises at least one predetermined bit in a predetermined byte of the configuration data.

[0307] 37. A method according to any of clauses 28 to 36, the method comprising performing byte-wise processing on the configuration data.

[0308] 38. A method comprising: [0309] generating an image configuration message comprising a header part and a payload part, the payload part comprising a set of image configuration parameters, the set of image configuration parameters relating to the processing of residual data; [0310] outputting the image configuration message, [0311] wherein the image configuration message corresponds to a given image only in a set of images.

[0312] 39. A method according to clause 38, wherein the set of image configuration parameters comprises a residual use parameter, wherein a value of the residual use parameter specifies whether or not input data for the given image is to be reconstructed using residual data for the given image.

[0313] 40. A method according to clause 39, wherein the set of image configuration parameters comprises quantisation data indicative of a quantisation level of data useable reconstruct, for the given image, the input data using upsampled data.

[0314] 41. A method according to clause 39 or clause 40, wherein the set of image configuration parameters comprises a picture type parameter that specifies whether the given image is progressive or interlaced.

[0315] 42. A method according to any of clauses 38 to 41, the method comprising generating and/or outputting a different image configuration message for each image in the set of images.

[0316] 43. A method comprising: [0317] receiving an image configuration message comprising a header part and a payload part, the payload part comprising a set of image configuration parameters, the set of image configuration parameters relating to the processing of residual data; [0318] performing image reconstruction using the received image configuration message, [0319] wherein the image configuration message corresponds to a given image only in a set of images.

[0320] 44. A method according to clause 43, wherein the set of image configuration parameters comprises a residual use parameter, wherein a value of the residual use parameter specifies whether or not input data for the given image is to be reconstructed using residual data for the given image.

[0321] 45. A method according to clause 43, wherein the set of image configuration parameters comprises quantisation data indicative of a quantisation level of data useable reconstruct, for the given image, the input data using upsampled data.

[0322] 46. A method according to clause 44 or clause 45, wherein the set of image configuration parameters comprises a picture type parameter that specifies whether the given image is progressive or interlaced.

[0323] 47. A method according to any of clauses 43 to 46, the method comprising receiving a different image configuration message for each image in the set of images.

[0324] 48. A method comprising: [0325] generating an image set configuration message comprising a header part and a payload part, the payload part comprising a set of image set configuration parameters, the set of image set configuration parameters relating to the processing of residual data; [0326] outputting the image set configuration message, [0327] wherein the image set

configuration message corresponds to multiple images in a set of images.

[0328] 49. A method according to clause 48, wherein the set of image set configuration parameters comprises a transform type parameter relating to a transformation operation performed on residual data.

[0329] 50. A method according to clause 48 or clause 49, wherein the set of image set configuration parameters comprises an upsample type parameter relating to an upsampling operation performed.

[0330] 51. A method according to any of clauses 48 to 50, wherein the set of image set configuration parameters comprises a depth type parameter relating to a bit depth associated with residual data.

[0331] 52. A method according to any of clauses 48 to 51, wherein the set of image set configuration parameters comprises a resolution type parameter specifying a resolution of input data for a given image in the set of images.

[0332] 53. A method according to clause 52, wherein a value of the resolution type parameter has a fixed bit-length.

[0333] 54. A method according to clause 53, wherein the value of the resolution type parameter has a bit-length of seven bits.

[0334] 55. A method according to any of clauses 52 to 54, wherein the set of image set configuration parameters comprises a custom resolution parameter relating to a resolution of the input data for the given image in the set of images.

[0335] 56. A method comprising: [0336] receiving an image set configuration message comprising a header part and a payload part, the payload part comprising a set of image set configuration parameters, the set of image set configuration parameters relating to the processing of residual data; [0337] performing image reconstruction using the received image set configuration message, [0338] wherein the image set configuration message corresponds to multiple images in a set of images.

[0339] 57. A method according to clause 56, wherein the set of image set configuration parameters comprises a transform type parameter relating to a transformation operation performed on residual data.

[0340] 58. A method according to clause 56 or clause 57, wherein the set of image set configuration parameters comprises an upsample type parameter relating to an upsampling operation performed.

[0341] 59. A method according to any of clauses 56 to 58, wherein the set of image set configuration parameters comprises a depth type parameter relating to a bit depth associated with residual data.

[0342] 60. A method according to any of clauses 56 to 59, wherein the set of image set configuration parameters comprises a resolution type parameter specifying a resolution of input data for a given image in the set of images.

[0343] 61. A method according to clause 60, wherein a value of the resolution type parameter has a fixed bit-length.

[0344] 62. A method according to clause 61, wherein the value of the resolution type parameter has a bit-length of seven bits.

[0345] 63. A method according to any of clauses 60 to 62, wherein the set of image set configuration parameters comprises a custom resolution parameter relating to a resolution of the input data for the given image in the set of images.

[0346] 64 Apparatus configured to perform a method according to any of clauses 1 to 63.

[0347] 65. A computer program comprising instructions which, when executed, cause an apparatus to perform a method according to any of clauses 1 to 63.

[0348] 66. A computer-readable medium comprising a computer program according to clause 65.

Claims

1-23. (canceled)

24. A method comprising: obtaining a configuration block comprising a configuration value, the configuration block being useable to reconstruct a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy; and obtaining an extension configuration block comprising an extension configuration value, wherein the configuration value and/or the extension configuration value is useable by a decoder to reconstruct the first representation of the image at the first level of quality.

25. The method according to claim 24, wherein the extension configuration value is indicative of one or more predetermined actions to be performed by the decoder in relation to the configuration value.

26. The method according to claim 25, wherein the one or more predetermined actions comprise deriving an updated configuration value by updating the configuration value based on the extension configuration value, the updated configuration value being useable by the decoder to reconstruct the first representation of the image at the first level of quality.

27. The method according to claim 26, wherein the updating of the configuration value comprises adjusting the configuration value based on the extension configuration value.

28. The method according to claim 27, wherein the updating of the configuration value comprises incrementing the configuration value based on the extension configuration value.

29. The method according to claim 25, wherein, when the extension configuration value has one or more first predetermined extension configuration values, the one or more predetermined actions comprise using the configuration value to reconstruct the first representation of the image at the first level of quality.

30. The method according to claim 25, wherein, when the extension configuration value has one or more second predetermined extension configuration values, the one or more predetermined actions comprise using the extended configuration value to reconstruct the first representation of the image at the first level of quality.

31. The method according to claim 24, wherein the configuration value and/or the extension configuration value is indicative of a type of upsampler to be used to reconstruct the first representation of the image at the first level of quality.

32. The method according to claim 24, wherein the configuration value and/or the extension configuration value is indicative of a type of filter to be used to reconstruct the first representation of the image at the first level of quality.

33. A The method according to claim 24, wherein the configuration value and/or the extension configuration value is indicative of a temporal setting to be used to reconstruct the first representation of the image at the first level of quality.

34. The method according to claim 24, wherein the obtaining of the configuration block comprises generating the configuration block and wherein the obtaining of the extension configuration block comprises generating the extension configuration block.

35. The method according to claim 24, wherein the obtaining of the configuration block comprises receiving the configuration block and wherein the obtaining of the extension configuration block comprises receiving the extension configuration block.

36. The method according to claim 24, wherein the configuration block is useable by first and second types of decoder to reconstruct the first representation of the image at the first level of quality, wherein the extended configuration block is useable by the second type of decoder to reconstruct the first representation of the image at the first level of quality, and wherein the extended configuration block is not useable by the first type of decoder to reconstruct the first representation of the image at the first level of quality.

37. The method according to claim 36, wherein the second type of decoder is a newer type of decoder than the first type of decoder.

38. An encoder configured to: generate a configuration block comprising a configuration value, wherein the configuration value indicates a configuration to be selected by a decoder; and generate an extension configuration block comprising an extension configuration value, wherein the extension configuration value indicates one or more predetermined actions to be performed by a decoder in relation to the selected configuration.

39. A decoder configured to: obtain a configuration block comprising a configuration value; select a configuration from a plurality of configurations based on the configuration value; obtain an extension configuration block comprising an extension configuration value; perform one or more actions in relation to the selected configuration based on the extension configuration value; and reconstruct, using the configuration block, a first representation of an image at a first level of quality in a tiered hierarchy having multiple levels of quality using a second representation of the image at the first level of quality, the second representation being based on a representation of the image at a second, lower level of quality in the tiered hierarchy, wherein the decoder uses the extension configuration value to reconstruct the first representation of the image at the first level of quality.
